# APP – teorie

1. Excludere mutuala bazat pe XCHG

**Instrucţiunea XCHG(r,m) interschimbă, ca o operaţie atomică, conţinutul registrului r cu celula de memorie m (semafor). ❑ deci ri este o variabilă locală a sarcinii Si şi este iniţializată cu 0, iar ❑ m este o variabilă globală, comună tuturor sarcinilor care este iniţializată cu 1.**

**Ambele variabile pot lua valorile 0, 1. Algoritmul pentru Si va fi:**

**<inceput sarcina>;**

**Repetă**

**XCHG(ri ,m);**

**până_când ri;**

**<sectiune critica>;**

**XCHG(ri ,m);**

**<rest sarcina> ;**

**Numai procesul care găseşte m=1 va intra în secţiunea critică şi va trece în zero. La ieşire din secţiunea critică va returna pe m la valoarea 1 pentru a permite celorlalte sarcini să intre în secţiunea critică. ( m=1 este o conventie, in unele sisteme se considera m=0) . Când o sarcina Si intră în secţiunea critică, ri=1.**

2. algoritmul optim de partiție a unui microprogram descries prin graf de dependențe de date vs algoritmul euristic de partiție

In cadrul algoritmului optim de partitie a unui microprogram, descris prin graf de dependente de date, se va realiza partitia optima prin generarea din setul de microoperatii uODk a tuturor uIC posibile, pentru fiecare dintre ele se va analiza influenta pe care o are asupra generarii setului de microoperat ii disponibile urmator, uODk + 1. In cazul algoritmului euristic de partitie, el este bazat pe ordonare a microoperatiilor din setul disponibil uODk in functie de succesorii in graful de dependente de date. In cadrul

algoritmului euristic nu se va genera insa o partitie optima, dar va fi mult mai rapida in unele situatii. partitie euristica = bazata pe ordonarea operatiilor in functie de succesori -> complexitate liniara algoritm optim de partitie = NP completa

3. Descrieţi principiul de funcţionare a sistemelor de tip Data Flow . Comentaţi componentele principale ale arhitecturii.

   Sistemele de tip Data Flow au un model ce este bazat pe fluxul de date, in cadrul caruia executia unei instructiuni de prelucrare depinde de disponibilitatea operandului. In cadrul acestor sisteme nu avem Program Counter. Specifica efectuarea unor operatii imediat ce vom avea operanzii disponibili si din aceasta cauza nu avem nevoie de contorul de program. De asemenea, nu este nevoie de adrese pentru memorarea variabilelor iar operatia se efectueaza numai atunci cand token-ul este prezent pe ambele intrari ale operandului. Datele se stocheaza in cadrul instructiunilor, adica vom mentine un token cu valoare binara in cadrul instructiunii pentru fiecare data. O unittate specializata va verifica disponibilitatea datelor. Ca avantaje avem faptul ca avem un paralelism ridicat si o viteza de executie mai mare. De asemenea este usor de implementat partea de comunicatie si sincronizare, insa vom avea overhead mare pentru control, iar manipularea structurilor de date va fi dificila. Avem Token = {Data, Tag Destinatie, marker} si Match = {Tag, Destinatie}. Avem Df1, Df2, …, Dfn ce reprezinta retea de masini de baza ce sunt interconectate printr-o retea de comutare; Token Queue cu care pe masura ce vom avea token-uri, ele vor fi adaugate in coada Matching Unit - elementul de prelucrare are un token pe fiecare ramura; Instruction store - ia elementele de prelucrare si le pune pe o unitate de procesare

4. Exemple de calculatoare paralele

   (aici la ce se referă? Taxonomia Flynn cu extinderea pe tipuri de arhitecturi paralele?)

5. Teorema de suficienta + demonstratie
   **Sistemele de sarcini formate din sarcini mutual neinterferente sunt determinate**

**Justificare (prin inducţie)**

- **Pentru un sistem de sarcini format dintr-o singura sarcina este evident.**
- **Presupunem că afirmaţia este adevărată pentru sisteme cu n-1 sarcini, (α'1 şi α'2 sunt secvenţe de execuţie pentru un sistem cu n-1 sarcini)**
- **Presupunem sistemul C = (S, <' fiind obţinută din < eliminând relaţia de precedenţă ce implică pe S. V(Mi , α'1 ) = V(Mi , α'2 ) din ipoteza de inducţie.**
- **Deci se poate presupune că valorile din DS sunt aceleaşi, atât pentru α'1 cât şi pentru α'2 ,**
- **respectiv F(Mi , α'1 ) = F(Mi , α'2 ) pentru Mi  DS .**

- **Rezultă astfel că pentru α'1 şi α'2 , sarcina S scrie aceeaşi valoare v pentru orice celulă Mi RS . Pentru Mi RS : V(Mi , α1 ) = V(Mi , α'1 SI SF) Lemă = V(Mi , α'1 ) Mi RS si ip. de inducţie = V(Mi , α'2 ) Mi RS = V(Mi , α'2 SI SF) Lemă = V(Mi , α2 ) Pentru Mi RS V(Mi , α1 ) = V(Mi , α2 ) Pentru Mi RS : V(Mi , α1 ) = V(Mi , α'1 SI SF) Lemă = (V(Mi , α'1 ),v) ip. de inducţie = (V(Mi , α'2 ),v) = V(Mi , α'2 SI SF) Lemă = V(Mi , α2 ) Pentru Mi RS V(Mi, α1) = V(Mi, α2)**
- **Sistemul C este determinat dacă sarcinile sunt mutual neinterferente.**

6. Limita inferioara

**Limita inferioară =** $\mu$**, limita inferioară pentru situaţia în care resursele sunt distincte.**

$\mu$ **=** $max$ **{|**$\mu oPi$**|} 1 ≤** $i$ **≤** $P$

**Fiind o singur resursa, se va fi controlata secvenţial, deci avem |**$\mu oPi$**|, fiind |**$\mu oPi$**| microoperatii.**

**Când resursele nu ar fi fost diferite, am putea considera că**

$\mu$ **=** $max$ **{[|**$\mu oPi$**| / |**$Pi$**|]}**

**În calculul acestei limite, s-a presupus că nu există dependenţă de date între microoperatiile care controleaza diferite resurse.**

Eu cred ca aici nu vrea asta, ci mai degraba sa ii explici la ce se refera limita asta inferioara, eventual pe scurt cazurile respective. Si sa ii mai explici si de ce vrei sa afli limita asta inferioara. Chiar nu cred ca vrea formulele si explicatii la formule.

7. Sun-Ni

**Introduce modelul bazat pe limita de memoriei**

- **Ts = fseq + g(p)(1- fseq)**
- **unde G(n) este cresterea de memorie a unui procesor**
- **Tp = fseq + g(p)(1- fseq) /p**
- **Vp=Ts/Tp= (fseq + g(p)(1- fseq)) /(fseq + g(p)(1- fseq) /p)**
- **Caz 1 : g(n)=1**
- **Vp= (fseq + 1- fseq) /(fseq + (1- fseq) /p)= 1 /(fseq + (1- fseq) /p) ~ Amdahl**
- **Caz 2 : g(n)=p**
- **Vp=Ts/Tp= (fseq + p(1- fseq)) /(fseq + p(1- fseq) /p)=(fseq + p(1- fseq)) /1 = fseq + p(1- fseq) ~ Gustafson**
- **Caz 3: g(p)= m > p**
- **Vp=Ts/Tp= (fseq + m(1- fseq)) /(fseq + m(1- fseq) /p)=**
- **(fseq + m(1- fseq))/(m/p+ (1-m/p)fseq)**

8. Sisteme de clasa determinate, secventa de executie partiala

9. Clase compatibile. Clase incompatibile

10. Modalitati de codificare microinstructiuni. Avantaje si dezavantaje pentru fiecare

11. Metode de calcul paralel: enumerare + comparatie intre memoria partajata si procesare sistolica

12. Amdhal + Worldon + Gustafson

13. Inidicatori paraleli.
    **Viteza de prelucrare – Vp = T1 / Tp**

**Eficienta - Ep = Vp / p**

**Redundanta - Rp = Op / O1**

**Utilizarea - Up = Op / (p * Tp) <= 1**

14. Sistem de sarcini determinat. Secvente de executie. Secventa partiala de executie.

    Secventa de executie: Alfa = a1a2a3…ai, ai+1, … a_2n. ai = eveniment ce declanșează sau termina o sarcina, cu respectarea ordinii de precedenta < (< relatie de ordine partiala).

    Secventa de executie partiala: orice prefix dintr-o secventa de executie valida.

    Sistemul de sarcini este determinat daca indiferent de ordinea de executie a sarcinilor independente (adica pentru orice secventa alfa valida) si indiferent de timpul necesar unei sarcini independente, se produce acelasi rezultat si, in plus, daca acest lucru se respecta pentru orice interpretare f_s a sistemului. (f_s = ceea ce executa fiecare sarcina).

** E corect si ce scrie mai jos, dar e dintr-un curs de mai tarziu. Deci le-as zice pe amandoua.

Un sistem de sarcini C este determinat dacă pentru orice stare iniţială s0 dată, V(Mi ,α) = V(Mi ,α'), 1 i m, pentru toate secvenţele de execuţie α ,….α' din C.

15. Tipuri de micoinstructiuni

**microinstrucţiuni operaţionale** care controlează primitivele funcţionale ale unităţii de execuţie a sistemului numeric, asigurând fluxul de informaţie şi acţiunile asupra resurselor;

**microinstrucţiune de ramificaţie** (de salt) care inspectează starea primitivelor funcţionale şi asigură ramificaţia în algoritmul de control, constituind suportul pentru implementarea deciziilor.

16. Sisteme microprogramate vs. microprogramabile

17. Algoritmul de partitionare (in 8 pasi).

**P1. Initializare**

**Se initializeaza partitia** $\mu PT = \Phi$ **,** $\mu PTA = \mu B(\mu O)$ **- tot microsublocul (deci, daca le luam si le facem sevcential)**

$\mu OD$ **=** $\{\mu Oj \mid \forall \mu Oj \in \mu B(\mu O)$ $nu\ exista\ \mu Oi < \mu Oj\}$ **- deci pentru care nu exista predecesori**

$\mu OND = \{\mu Oi \mid \mu Oi \in \mu B(\mu O) \setminus \mu OD\}$ **- operatii nedisponibile**

**P2. Daca** $\mu OND = \Phi\ si\ \mu OD \leq 3$ **atunci salt la pas 5**

**P3. Se genereaza {IC} – multimea de microoinstructiuni complete**

$\{\mu IC\} = \{\mu ICj , . . . , \mu ICk\}$ **- din OD curent daca j ≠ k (s-au generat mai multe IC) atunci salvare PTj = PT ODj = setul curent OD {IC}j = {IC} \ ICj**

**P4.** $\mu PT = \mu PT \cup \mu ICj\ \mu OD = \mu OD \setminus \mu ICj \cup \{\mu Od\}j\ \mu OND = \mu OND \setminus \{\mu Od\}j$

**daca** $\mu OD \neq 0\ si\ \mu PT \leq \mu PTj - 1$ **atunci salt pas 2**

**P5. daca OD = atunci salt pas 7**

**P6. Se genereaza µIC din µOD curent. µPT = µPT U µIC; µOD = µOD \ µIC;**

**P7. daca μOD ≠ Φ si |μPT|< | μPTA| atunci μPTA = μPT altfel daca |μPTA| ≤ nmμI atunci μPTA este optima STOP.**

**P8. daca {$\mu IC$}$j$ ≠ 0 (cel care a fost salvat la pasul 3) atunci reface $\mu PT$ = $\mu PTj$ ; $\mu OD$ = $\mu ODj$ ; $j \leftarrow j$ + 1; (trec la urmatorul, selecteaza urmatoarea IC din setul generat) $\mu IC$ = $\mu ICj$ $\mu OND$ = {$\mu Oj$ |$\mu Oj \in \mu B(\mu O) \setminus (\mu PT \cup \mu OD)$} salt pas 4 altfel PTA este optima .**

20. Fie un procesor care implementează o structura paralela pipeline de citire interpretare executie pentru procesare suprascalară , cu patru unitati paralele. Presupunând un ciclu de instrucțiuni contine:

-citire care necesita o singura perioada de ceas

-decodificarea instructiunii necesita două perioade de ceas

-executia instructiunii necesita doua perioade de ceas Avem o secvență de 3000000 de instrucțiuni. Considerand freventa ceasului in Ghz de 1. Calculati durata de executie a secventei de program in milisecunde

(1 + 2 + 3*10^6 / 4 * 2) * 10^(-9) ~= 1.5ms

Considerând că în urma analizei microoperațiilor dintr-un subloc au rezultat microintrucțiunile complete,mIC1,…mIC5 care conțin micro[1]operațiile ca în tabelul alăturat:

mIC1 = mo1 mo2 mo3 mo4 mo5 mo6

mIC2 = mo3 mo7 mo8 mo9

mIC3 = mo1 mo2 mo8 mo9 mo10

mIC4 = mo4 mo8 mo11

mIC5 = mo6 mo8

Care este organizarea optimă a câmpurilor din formatul general al microinstrucțiunilor:

a. Câmp 1: (mo1)

Câmp 2: (mo2)

Câmp 3: (mo3)

Câmp 4: (mo4 mo7 mo9 mo10 mo11)

Câmp 5: (mo5)

Câmp 6: (mo6)

Câmp 7: (mo8)

b. Câmp 1: (mo1)

Câmp 2: (mo2)

Câmp 3: (mo3)

Câmp 4: (mo4 mo9 mo10)

Câmp 5: (mo5 mo11)

Câmp 6: (mo6 mo7)

Câmp 7: (mo8)

c. Câmp 1: (mo1)

Câmp 2: (mo2)

Câmp 3: (mo3)

Câmp 4: (mo4 mo9)

Câmp 5: (mo5 mo11)

Câmp 6: (mo6 mo7 mo10)

Câmp 7: (mo8)

**Ar fi D, dar dc? --** n-am idee, din ce văd și C este validă, nu văd ce ar fi greșit la ea **-> se calculeaza si costul**: pt fiecare camp, numeri cate instructiuni are, adaugi 1 (o instructiune nop imaginara, la fiecare camp), si obtii N -> te gandesti de cati biti ai nevoie pt a reprezenta N numere (ex: N = 4 => 2 biti, N = 5 => 3 biti); aduni toti bitii necesari pt toate campurile si obtii **costul**. Se alege varianta cu **costul minim** (exemple: campul 1 de mai jos are 1+1 instructiuni, e suficient 1 bit; campul 5 de mai jos are 3+1 instructiuni, sunt suficienti 2 biti - se va observa ca varianta **c** are costul **10**, iar **d** are costul **9**)

**d**. Câmp 1: (mo1)

Câmp 2: (mo2)

Câmp 3: (mo3)

Câmp 4: (mo4)

Câmp 5: (mo5 mo7 mo10)

Câmp 6: (mo6 mo9 mo11)

Câmp 7: (mo8)

1. Legile Amdahl, Worlton, Gustafson si Sun-Ni
2. Limita inferioara pentru nivele de alocare a sistemelor de sarcini
3. Modele de calcul paralel (comparatii: sistolic, data flow, transfer mesaje)
4. Sistem de sarcini determinat, nedeterminat si secventa partiala de executie
5. Pasii generali ai algoritmului de repartizare microoperatii in microinstructiuni complete
6. Teorema de necesitate cu demonstratie
7. Teorema de suficienta cu demonstratie
8. Clasa de compatibilitate + incompatibilitate + cost + cum se determina
9. Algoritm euristic + optim de impartire pe niveluri
10. Dati exemple de sisteme cu arhitectura paralela
11. Modele sistolic (memorie partajata cu transfer de mesaje)
12. Concatenarea sistemelor de sarcini
13. Sisteme echivalente
14. Structura unei unitati de comanda microprogramate
15. Indicatori de performanta (prelucrari paralele)
16. Diferenta intre sistemele microprogramate si microprogramabile
17. Modalitati de implementare a memoriei de control
18. Organizarea microoperatiilor
19. Legatura dintre algoritmi paraleli si arhitecturi paralele

## 1. Legile Amdahl, Worlton, Gustafson si Sun-Ni

### Amdahl

Stabileste o limită a creșterii de viteză a structurilor paralele în raport cu structurile monoprocesor

$R_H$ = rata de execuție pe structura paralelă

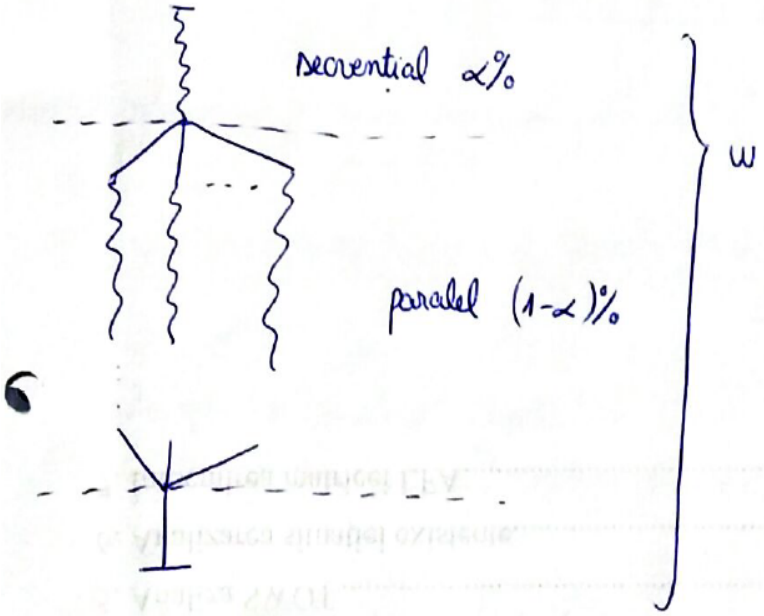
$R_L$ = rata de execuție pe structura monoprocesor

$f$ = procentul secvenței de instrucțiuni executată în paralel

$1-f$ = procentul —||— secvențial

Formula de bază: $R = \frac{1}{\frac{f}{R_H} + \frac{1-f}{R_L}}$

Formă redusă a formulei:




! Dimensiunea problemei rămâne constantă cu creșterea dimensiunii sistemului.

! Porțiunea secvențială a problemei limitează speed-up-ul total care poate fi atins în ciuda creșterii resurselor.

Creșterea de viteză:

$$V_p = \frac{T_1}{T_p} = \frac{w}{\alpha w + \frac{(1-\alpha)w}{p}} = \frac{1}{\alpha + \frac{1-\alpha}{p}}$$

Indiferent de numărul de procesoare, creșterea de viteză este limitată de caracteristicile intrinseci ale programului.

**Worlton**

Un program paralel constă din
- o secțiune secvențială
- o secțiune paralelă
- o secțiune de sincronizare

$\Rightarrow$
- $t_s$ = timp sincronizare
- $t_o$ = timp overhead
- $t$ = timp mediu pt. 1 task
- $N$ = nr. task-uri
- $P$ = nr. procesoare

Timpul pentru un procesor: $T_1 = Nt$

Timpul pentru $P$ procesoare: $T_{Np} = t_s + \left\lceil \frac{N}{P} \right\rceil \cdot (t + t_o)$

Creșterea de viteză:

$$V_p = \frac{T_1}{T_{Np}} = \frac{Nt}{t_s + \left[\frac{N}{p}\right](t+t_o)} = \frac{1}{\frac{t_s}{Nt} + \frac{1}{N}\left[\frac{N}{p}\right]\left(1+\frac{t_o}{t}\right)}$$

$\frac{t_s}{Nt}$ ~ sincronizare; $\frac{t_o}{t}$ ~ overhead

Pentru rezultate optime:

- $p$ divizor al lui $N$
- $\frac{t_s}{N \cdot t}$ cât mai mic → $t_s$ cât mai mic
  → mărirea intervalului între sincronizări
- $\frac{t_o}{t}$ cât mai mic → $t_o$ cât mai mic
  → creșterea granularității task-urilor

## GUSTAFSON

← Dimensiunea problemei scalează până la o limită fixă a timpului

- A arătat că oamenii nu sunt interesați de rezolvarea unei probleme în cel mai scurt timp (Amdahl), ci mai degrabă de rezolvarea problemei celei mai mari într-un timp rezonabil
- Folosește n procesoare pentru rezolvarea unui task

$$w' = \alpha w + (1-\alpha) w n \quad \text{unde} \begin{cases} \alpha = \text{procentul de execuție secvențială} \\ 1-\alpha = \text{procentul de execuție paralelă} \end{cases}$$

Creșterea de viteză:

$$V_p = \frac{T_1}{T_n} = \frac{\alpha w + (1-\alpha) w n}{\alpha w + \frac{(1-\alpha) w n}{n}} = \frac{\alpha w + (1-\alpha) w n}{\alpha w + (1-\alpha) w} = \frac{\alpha + (1-\alpha) n}{\alpha + (1-\alpha)} = \alpha + (1-\alpha) n$$

- Sugerează că e benefică construirea unui sistem paralel mare, pt. că speed-up-ul poate crește liniar cu dimensiunea sistemului doar dacă se menține același timp fix de execuție.

# SUN & NI

- Dimensiunea problemei scalează până la o limită setată de dimensiunea memoriei sistemului
- generalizare pentru Amdahl și Gustafson → când $G(n)=1 \Rightarrow$ Amdahl
  → când $G(n)=n \Rightarrow$ Gustafson

$$w = \underbrace{\alpha w}_{\text{componenta secvențială}} + (1-\alpha)\, w \cdot G(n)$$

↖ câștigul pe care îl avem prin adăugarea memoriei

Creșterea de viteză:

$$V_p = \frac{T_1}{T_n} = \frac{\alpha w + (1-\alpha) w \cdot G(n)}{\alpha w + \frac{(1-\alpha) w \cdot G(n)}{n}} = \frac{\alpha + (1-\alpha)\, G(n)}{\alpha + \frac{(1-\alpha)\, G(n)}{n}}$$

Caz A : nu adăugăm memorie suplimentară $\Rightarrow G(n)=1$

$$V_p = \frac{\alpha + (1-\alpha)}{\alpha + \frac{1-\alpha}{n}} = \frac{1}{\alpha + \frac{1-\alpha}{n}} = \text{Amdahl}$$

Caz B : $G(n) = n$

$$V_p = \frac{\alpha + (1-\alpha) n}{\alpha + \frac{(1-\alpha) n}{n}} = \alpha + (1-\alpha) n = \text{Gustafson}$$

## 2. Limita inferioara pentru nivele de alocare a sistemelor de sarcini

CALCULUL LIMITEI INFERIOARE PENTRU PROCESUL DE ALOCARE

Microsubbloc:

$\mu SB = (\mu B(\mu_o), \alpha)$

$\mu B = \{\mu_{o_1}, \dots, \mu_{o_{|B|}}\}$

$P = \{P_1, \dots, P_{|P|}\}$ ← elemente de execuție (procesoare)

$\mu_{o_i}$ acționează asupra lui $P_j$.

Limita inferioară a nivelelor de alocare

CAZ 1 Calculăm înălțimea grafului de dependență de date $h$

Calea cea mai lungă în G de la nodul inițial către nodul final

CAZ 2: Calculăm $\rho$

Ținem cont de distribuția pe procesoare

⇒ Împărțim graful: $G = \{G_1, \dots, G_{|P|}\}$ ⇔ identificăm subgrafurile din graful inițial ~~per~~ corespunzătoare fiecărui procesor

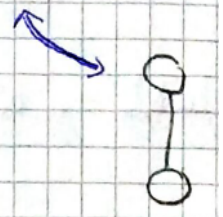
$G_i = \{\mu_o \mid \mu_o$ controlează $P_i$ / se execută pe $P_i\}$

Aceste subgrafuri ar putea să aibă mai multe sarcini?

Definim $SGM_{ij} \subset G_i$ cu proprietatea că: $\forall \mu_o \in SGM_{ij}$, atunci există o $\mu_{o_k}$ astfel încât:

$\mu_o \prec \mu_{o_k}$ SAU $\mu_{o_k} \prec \mu_o$

↓
subgraf maximal



$G_i = \| SGM_{ij}$ ⟺ combinare paralelă

$\mu_{o_T}$ ⇒ nu există $\mu_{o_i}$ astfel încât $\mu_{o_T} \prec \mu_{o_i}$

↑
Microoperație terminală

$\mu_{o_{T_j}}$ ⇒ microoperație terminală în $SGM_{ij}$

$\| \mu_{o_{T_j}} \|$ = poziția sarcinii terminale în graful G

↗ normă

Pentru fiecare $G_i$, calculăm $\rho_i$ (limita de nivel pentru fiecare subgraf). După, încercăm să calculăm $\rho$ folosind $\rho_i$ și sarcinile terminale din fiecare subgraf.

$1 < |\mu_{o_{T_{jk}}}| \leq N_{\mu_{oT}}$

↑ numărul de sarcini terminale

$$\rho_i = \max(\rho_{ij})$$

$$\rho_{ij} = |SGM_{ij}| + \min_k \| \mu_{o_{T_{jk}}} \|$$

$$\rho = \max_i \left( \max_j \left( SGM_{ij} + \min_k \left( \| \mu_{o_{T_{jk}}} \| \right) \right) \right)$$

↕ estimare a numărului minim a nr. de alocare

OBSERVAȚIE !

Dacă procesoarele nu sunt distincte, definim $\rho$ astfel:

$$\rho = \max_i \left( \max_j \left( \max \left( \frac{|SGM_{ij}|}{|P_i|}, h(SGM_{ij}) \right) + \min_k \left( \| \mu_{OT_{jk}} \| \right) \right) \right)$$

$h(SGM_{ij})$ ↑ înălțimea subgrafului maximal $ij$

$h(SGM_{ij})$ ↑ din cauza dependenței de date

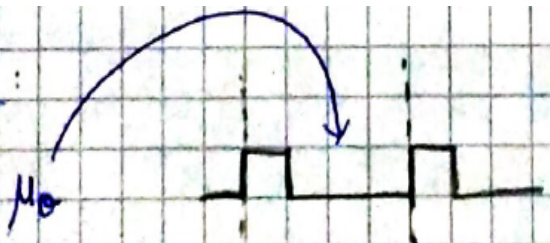
CAZ 3:

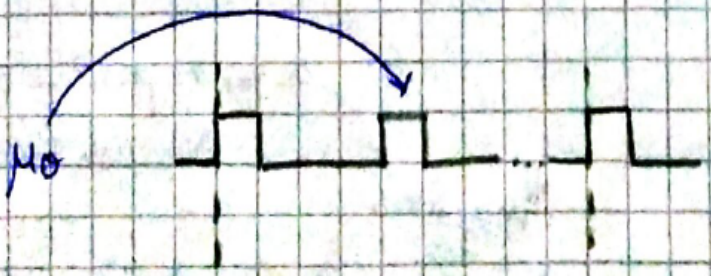
- nu considerăm dependențe de date

$$\mu = \max_i \left( \frac{\mu_B(\mu_O)_{P_i}}{|P_i|} \right)$$

$\mu_B(\mu_O)_{P_i}$ → numărul de operații asociate lui $P_i$

CAZ 4:




$z = \max_k (|\mu_{0mc_k}| \cdot n_k)$ unde $\mu_{0mc}$ ✓ microoperație multiple cycle

$n_k$ ↑ număr cicli

---

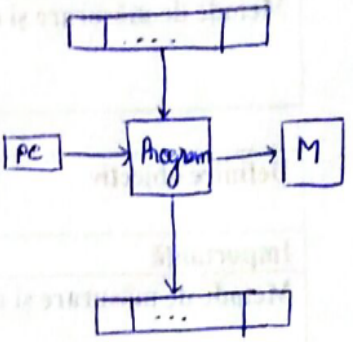
CONCLUZIE

Limita inferioară este: $\max(h, p, \mu, z) = Li$

**3. Modele de calcul paralel (comparatii: sistolic, data flow, transfer mesaje)**

## MODELUL RAM (RANDOM ACCESS MACHINE)

- Acest model este similar cu modelul mașinii Turing.
- Un astfel de model este utilizat pentru dezvoltarea de algoritmi și analiza complexității acestora, constituind punctul de plecare pentru modelele paralele
- Are
  - o bandă de intrare
  - o bandă de ieșire
  - program
  - program counter ⇒ folosit pentru execuția programului

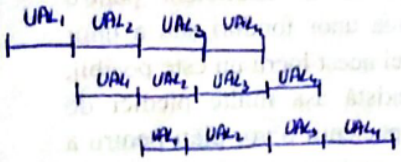


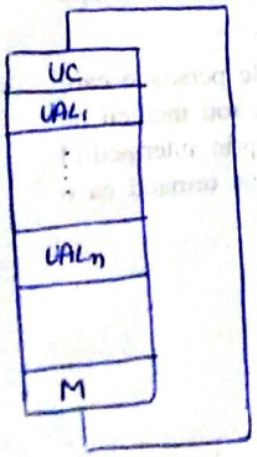

- RAM = $\{ S, \Gamma, \Sigma, f, s_0, Z \}$
  - $S$ – mulțime finită a stărilor
  - $\Gamma$ – alfabetul benzii
  - $\Sigma$ – alfabetul benzii de intrare
  - $f$ – funcție de tranziție: $f: S \times \Gamma \to S \times \Gamma$
  - $s_0$ – starea inițială ($s_0 \in S$)
  - $Z$ – mulțimea stărilor finale ($Z \subseteq S$)

## PIPELINE

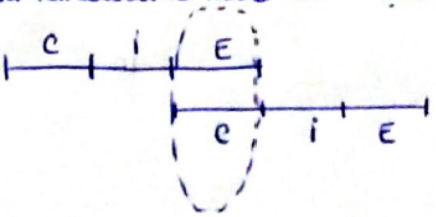
Presupune divizarea unui task T în subtask-uri $T_1, T_2, \dots, T_k$ și de a le atribui unor elemente de procesare.

Paralelismul se poate realiza la nivel de → prelucrare aritmetică
→ citire, interpretare, executare instrucțiuni



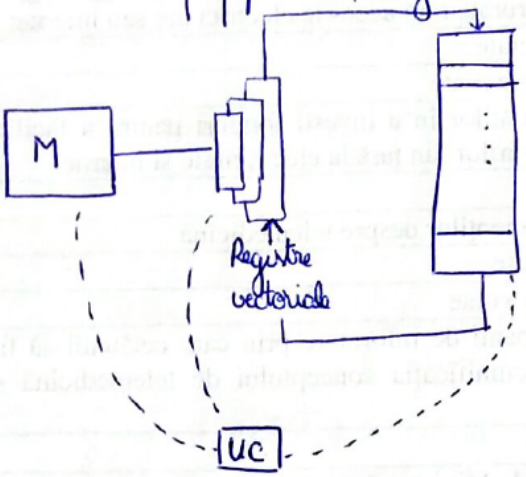




UC are mai multe faze: $UC_1, \dots, UC_m$
Când UC se ocupă de citirea, interpretarea și executarea instrucțiunilor mașină. După ce începe să execute, se poate apuca să citească o altă instrucțiune.




## PROCESOARE DE VECTORI

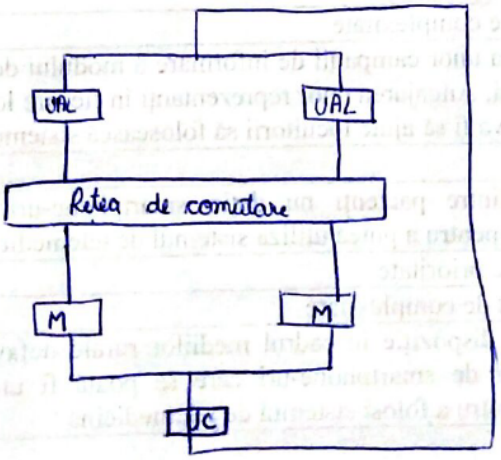
- există un set de instrucțiuni care tratează vectorul ca operand simplu
- există o UAL de tip pipeline și regiștri vectori (fiecare UAL se ocupă de un registru)




- poate fi asociată cu SIMD, dar spre deosebire de SIMD:
  - nu avem multiple unități de procesare
  - este o mașină secvențială, dar are mai multe UAL-uri

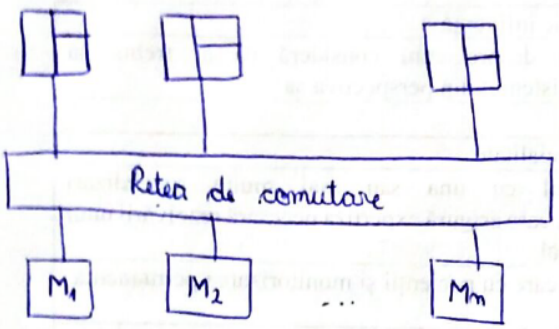
## PROCESARE MASIVĂ DE DATE

- Se referă la calculatoare paralele sincrone, care constau din UAL multiple, supervizate de o singură UC și care efectuează aceeași operație la un moment dat.
- UAL efectuează aceeași operație asupra unor date diferite
- poate fi asociat cu un SIMIS extins




## MEMORIE PARTAJATĂ

- Fiecare procesor conține propria unitate de prelucrare, propria memorie și propria unitate de comandă
- Se comunică prin intermediul unei rețele de comutare cu module de memorie partajată.
- Se încadrează la NUMA

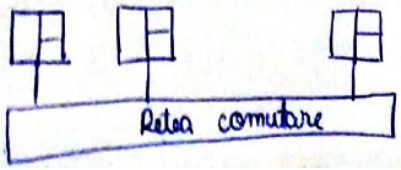


## PRAM - STRUCTURĂ PARALELĂ CU ACCES ALEATOR

- extensie a RAM
- comunicație între procesoare și memorie fără introducere de întârziere
- memorie comună disponibilă tuturor programelor
- 4 modalități de acces la memorie → EREW
  → CREW
  → ERCW
  → CRCW

## TRANSFER PRIN MESAJE

- Nu se mai partajează memoria
- Memoria este distribuită fiecărui procesor a.î. fiecare dispune de propriul program și propria memorie de date.
- Interconexiunea se realizează prin schimburi de mesaje prin intermediul unei rețele de comutare mesaje
- Exemplu: cluster (nu putem să vedem memoria celuilalt, dar folosim send și receive)

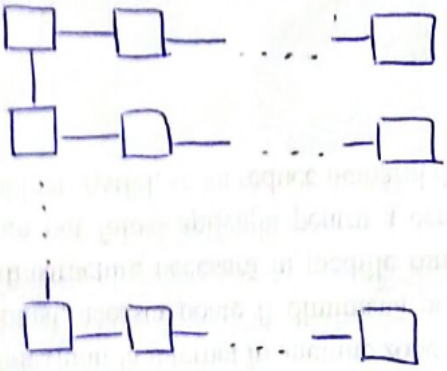


## PROCESARE SISTOLICĂ

- constă în elemente de procesare identice aranjate într-o structură pipeline

PA = procesor aritmetic

Pipeline linear

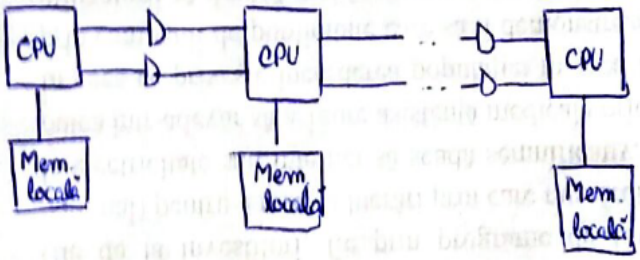


Pipeline matriceal

- Se caracterizează prin interconexiuni simple ce permite integrarea VLSI

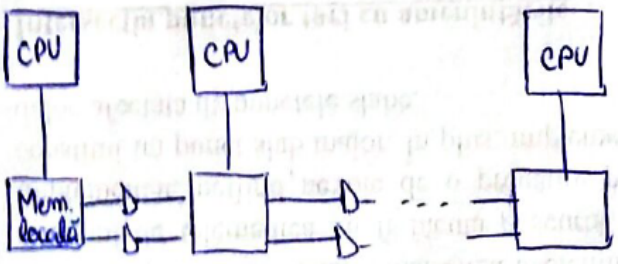
Metode de comunicare în sistemele sistolice:

(1)



După prelucrare, punem într-un buffer, iar următorul știe să ia de acolo.

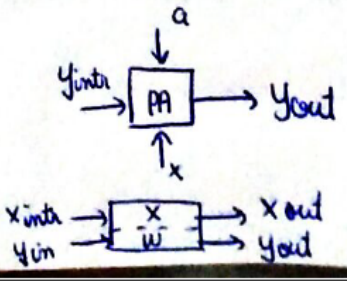
(2)




Prelucrez cevA N pun rezultatul în memorie. În acest moment există un acces direct de memorie care mută datele de ieșire în altă zonă de memorie ca fiind date de intrare ⇒ necesită implementarea unui DMA

Ex: $y = y_0 + Ax$

- ABORDARE 1: Secvențial ⇒ $O(n^2)$
- ABORDARE 2: Pipeline secvențial ⇒ Cost: $n \cdot PA$
  ⇒ $O(n)$
- ABORDARE 3: Pipeline matriceal ⇒ Cost: $n^2 \cdot PA$

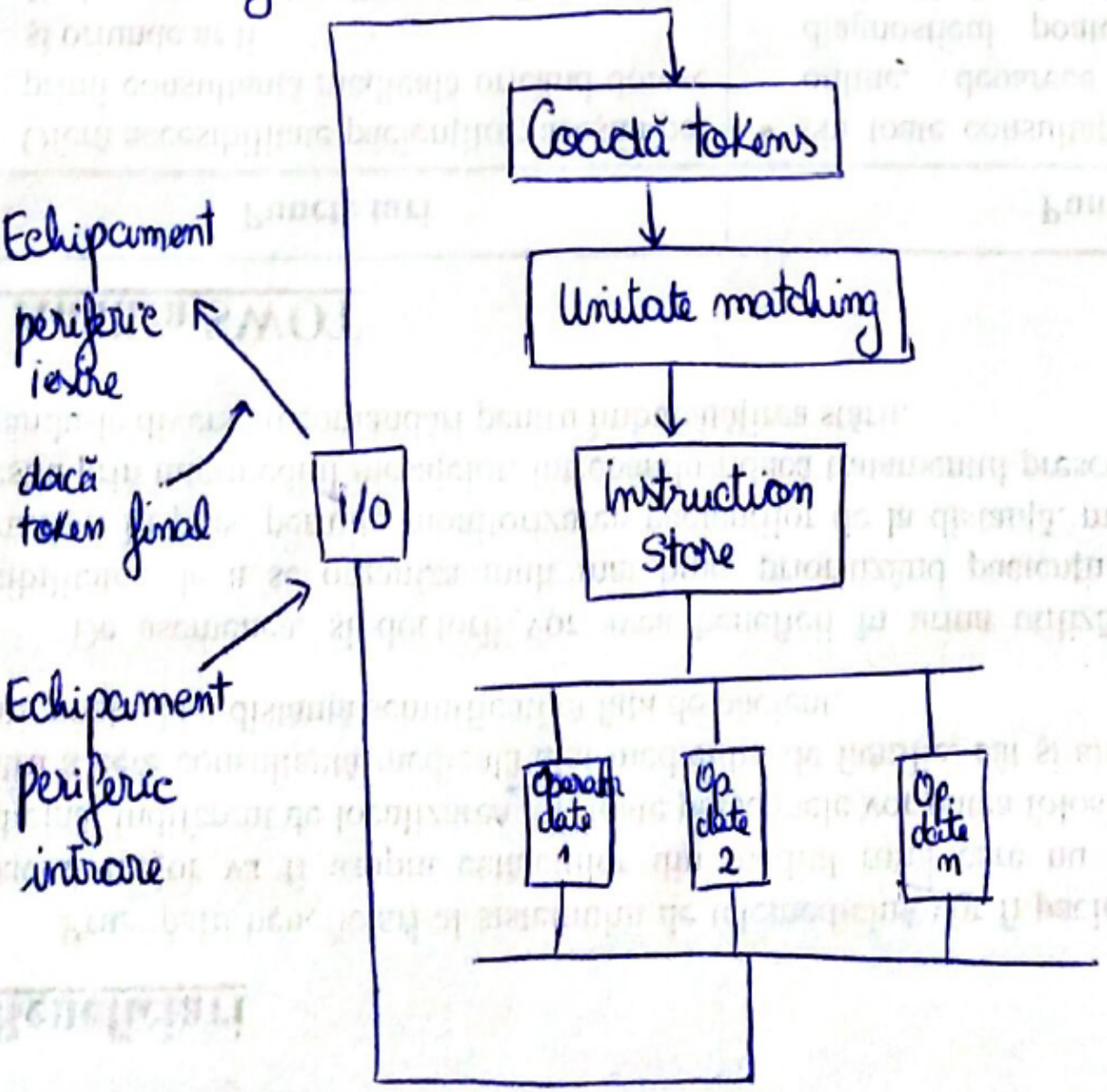


# DATA FLOW

- Specifică efectuarea unor operații imediat ce operanzii devin disponibili.
  ⇒ Se elimină necesitatea existenței contorului de program
- nu necesită adrese pentru memorarea variabilelor
- operația se efectuează numai când token-ul este prezent pe <u>ambele</u> intrări ale operandului
- datele se stochează în cadrul instrucțiunilor (se menține un token cu valoare binară în cadrul instrucțiunii pentru fiecare dată)
- disponibilitatea datelor este verificată de o unitate specializată, dedicată
- <u>Avantaje</u> → paralelism ridicat
  → viteză ridicată de execuție
  → ușor de implementat comunicația și sincronizarea
- <u>Dezavantaje</u> → overhead mare pentru control
  → manipulare dificilă a structurilor de date

- Token = { Data, Tag, Destinație, marker }

Match = { Tag, Destinație }



**4. Sistem de sarcini determinat, nedeterminat si secventa partiala de executie**

## SISTEM DE SARCINI

Fie $S = \{S_1, \dots, S_n\}$ o multime de sarcini, iar $\prec$ o relatie de ordine partială, nereflexivă, denumită relatie de precedență.

Perechea $C = (S, \prec)$ este un sistem de sarcini.

- $S_i \prec S_j \iff$ Pt. a începe și executa $S_j$, trebuie să se fi terminat $S_i$
- $S_i$ și $S_j$ sunt independente când $\prec$ dintre ele este $\emptyset$, adică $S_j \not\prec S_i$

## SECVENȚĂ DE EXECUȚIE

O secvență de execuție a unui sistem cu n sarcini $C = (S, \prec)$ este orice șir $\alpha$, $\alpha = a_1 a_2 \dots a_{2n}$ de evenimente de inițiere și terminare a proceselor cu respectarea relațiilor de precedență impuse de $\prec$.

$\bar{S}$ → inițiere sarcină

$\underline{S}$ → terminare sarcină

Proprietăți:

- $\alpha$ nu e unic
- pentru $\forall\ S \in S$, $\bar{S}$ și $\underline{S}$ apar o singură dată în $\alpha$
- dacă $a_i = \bar{S}$ și $a_j = \underline{S}$, atunci $i < j$
- dacă $a_i = \underline{S_k}$ și $a_j = \bar{S_p}$ cu $S_k \prec S_p$, atunci $i < j$

- **SECVENȚĂ DE EXECUȚIE PARȚIALĂ**

Este orice prefix al unei secvențe de execuție.

$\alpha_p = a_1 \dots a_p \;,\; p < 2n$

- **SPAȚIUL STĂRILOR** $\quad \sigma = s_0 s_1 \dots s_{2n}$

$s_0$ ↑ stare inițială

- **PROPRIETATEA DE DETERMINARE / NEDETERMINARE**

Un sistem de sarcini este considerat determinat dacă executându-se în paralel, respectând relațiile de precedență, indiferent de durata și ordinea de execuție a unor sarcini independente, rezultatul este același.

Un sistem de sarcini este considerat nedeterminat dacă rezultatele produse de sarcini independente depind de ordinea în care acestea se execută.

Nedeterminarea se poate rezolva introducând o relație de precedență între procesele care erau independente.

5. **Pasii generali ai algoritmului de repartizare microoperatii in microinstructiuni complete**

μSB = (μB(μo), <) microsublocul pe care trebuie sa il partiţionam pe niveluri (seturi de microinstrucţuni complete)
μPT = partiţia care se va genera
μPTA = partiţia de microinstructiuni anterioara, considerata cea mai buna pana in momentul respectiv
μoD = setul de micro-operatii disponibile la momentul curent
μoND = setul de micro-operatii nedisponibile la momentul curent
nmμI = numarul de micro-instructiuni complete care s-ar putea genera
μIC = micro-instructiunea completa generata din μoD
{μod} = multimea de micro-operatii disponibile rezultate ca urmare a generarii μIC

| | |
|---|---|
| Pas 1. | Initializare<br>$\mu PT = \Phi$<br>$\mu PTA = \mu B(\mu o)$<br>$\mu oD = \{ \mu o \mid \mu o \in \mu B(\mu o)$ nu exista $\mu o_i$ astfel ca $\mu o_i < \mu o\}$<br>$\mu oND = \mu B(\mu o) \setminus \mu OD$ |
| Pas 2. | Daca $\mu OND = \Phi$ si $\lvert\mu oD\rvert \leq 3$ atunci pas 5 |
| Pas 3. | Se genereaza<br>$\{\mu IC\}$ = multimea de $\mu I$ complete $\{\mu IC\} = \{\mu IC_j, \ldots, \mu IC_k\}$<br>Daca $j \neq k$ (s-a generat o singura $\mu IC$)<br>Atunci salvare<br>$\mu PT_j = \mu PT$;<br>$\mu ODj = \mu OD$ curent;<br>$\{\mu IC\}j = \{\mu IC\} \setminus \mu ICj$ |
| Pas 4. | $\mu PT = \mu PT + \mu IC_j$<br>$\mu OD = \mu OD \setminus \mu IC_j \cup \{\mu od\}_j$<br>$\mu oND = \mu OND \setminus \{\mu od\}_j$<br>Daca $\lvert\mu OD\rvert \neq 0$ si $\lvert\mu PT\rvert \leq \lvert\mu PT_j\rvert - 1$<br>atunci salt la Pas 2 |

| | |
|---|---|
| Pas 5. | Daca $\mu OD = \Phi$<br>Atunci salt Pas 7. |
| Pas 6. | Se genereaza<br>$\mu IC$ din $\mu OD$ curent<br>$\mu PT = \mu PT + \mu IC$<br>$\mu OD = \mu OD \setminus \mu IC$<br>Daca $\mu OD \neq \Phi$ si $\lvert\mu PT\rvert \leq \lvert\mu PTA\rvert - 1$<br>atunci salt Pas 4.<br>altfel salt Pas 2. |
| Pas 7. | Daca $\lvert\mu PT\rvert \leq \lvert\mu PTA\rvert$<br>atunci<br>altfel daca $\lvert\mu PT\rvert \leq$ nm$\mu$I<br>atunci $\mu PTA$ este optima STOP |
| Pas 8. | Daca $\{\mu IC\}j = 0$ (cel care a fost salvat la Pas 3.<br>Atunci reface $\mu PT = \mu PT_j$<br>$\mu OD = \mu Odj$<br>j=j+1 (selecteaza urmatoarea uIC din setul generat<br>$\mu IC = \mu IC_j$<br>$\mu oND = \mu B(\mu o) \setminus \mu PT \setminus \mu OD$<br>salt Pas 4.<br>Altfel $\mu PTA$ este optima STOP |

## 6. Teorema de necesitate cu demonstratie

**Enunt:**

Un sistem de sarcini C neinterpretat, dar la care stim domeniile de definitie si de valori este determinat pentru orice interpretare a sarcinilor sale numai dacă acestea sunt mutual neinterferente.

**Demonstratie:**

$C = (S, \prec)$

$S = \{s_1, \dots, s_n\}$

Demonstrăm prin reducere la absurd.

S și S' sunt independente, dar sunt interferente

Vrem să vedem dacă produc același rezultat.

$\alpha = \beta_1 \bar{S} \underline{S} \bar{S'} \underline{S'} \beta_2$

$\alpha' = \beta_1 \bar{S'} \underline{S'} \bar{S} \underline{S} \beta_2$

Considerăm celula $M_i$ de memorie care va fi folosită și de S și de S'

$M_i = R_S \cap R_{S'}$

$f_S \to u$

$f_{S'} \to v$

$V(M_i, \alpha) = V(M_i, \beta_1 \bar{S} \underline{S} \bar{S'} \underline{S'} \beta_2) = V(M_i, \beta_1 \bar{S} \underline{S} \bar{S'} \underline{S'}) = V(M_i, \beta_1)\, u\, v$

$V(M_i, \alpha') = V(M_i, \beta_1 \bar{S'} \underline{S'} \bar{S} \underline{S} \beta_2) = V(M_i, \beta_1 \bar{S'} \underline{S'} \bar{S} \underline{S}) = V(M_i, \beta_1)\, v\, u$

Deoarece de obicei $u \neq v \Rightarrow$ se produc rezultate diferite $\Rightarrow$ Trebuie ca $R_S \cap R_{S'} = \emptyset$

$M_j \in D_S \cap R_{S'}$

$M_i \in R_S$

$V(M_i, \alpha) = V(M_i, \beta_1 \bar{S} \underline{S} \bar{S'} \underline{S'} \beta_2) = V(M_i, \beta_1 \bar{S} \underline{S}) = V(M_i, \beta_1)\, u$ ← S' nu produce pt. că $\notin R_{S'}$

$V(M_i, \alpha') = V(M_i, \beta_1, \bar{S'} \underline{S'} \bar{S} \underline{S} \beta_2) = V(M_i, \beta_1 \bar{S'} \underline{S'} \bar{S} \underline{S}) = V(M_i, \beta_1)\, v$

$\Rightarrow D_S \cap R_{S'} = \emptyset$

Analog $D_{S'} \cap R_S = \emptyset$

$\Rightarrow$ Din toate cele 3 $\Rightarrow$ sistem neinterferent

**7. Teorema de suficienta cu demonstratie**

Enunț:

Sistemele de sarcini formate din sarcini mutual neinterferente sunt determinate

Demonstrație (prin inducție)

$C = (S, \leq)$

$S = \{s_1, \dots, s_n\}$

- PAS 1 : $n = 1$ (A)
- PAS 2 : Pp. că afirmația este adevărată pentru $n-1$ sarcini

$V(M_i, \alpha_1') = V(M_i, \alpha_2')$

↗ valorile lui $M_i$ în urma secvențelor $\alpha_1'$

- PAS 3 : Demonstrăm pentru $n$

Fie $\alpha_1', \alpha_2'$ secvențe de execuție pt. sistemul $C'$ cu $n-1$ sarcini.

$C' = (S \setminus s, \alpha')$

$\alpha_1 = \alpha_1' \bar{S} \underline{S}$

$\alpha_2 = \alpha_2' \bar{S} \underline{S}$

- $M_i \notin R_S$

↖ domeniul de valori

$V(M_i, \alpha_1) = V(M_i, \alpha_1' \bar{S} \underline{S}) = V(M_i, \alpha_1') = V(M_i, \alpha_2') = V(M_i, \alpha_2' \bar{S} \underline{S})$

din ipoteză ↓

$= V(M_i, \alpha_2)$

↖ nu produce mimic pt. că $M_i \notin R_S$

- $M_i \in R_S$

$V(M_i, \alpha_1) = V(M_i, \alpha_1' \bar{S} \underline{S}) = V(M_i, \alpha_1') \cup = V(M_i, \alpha_2') \cup = V(M_i, \alpha_2' \bar{S} \underline{S})$

$= V(M_i, \alpha_2)$

$\Rightarrow V(M_i, \alpha_1) = V(M_i, \alpha_2)$

## 8. Clasa de compatibilitate + incompatibilitate + cost + cum se determina

• **CLASA DE COMPATIBILITATE : CC**

Două microoperații $\mu o_i$ și $\mu o_j$ sunt compatibile dacă pentru orice K ($K \in [1, |\mu PT|]$) are loc următoarea relație:

dacă $\mu o_i \in \mu ic_K$, atunci $\mu o_j \notin \mu ic_K$ $\Leftrightarrow$ $\mu o_i$ și $\mu o_j$ nu se execută niciodată în paralel

$CC(\mu o) = \{\mu o \mid \forall \mu o$ și $\mu o_j$ sunt compatibile$\}$

↳ set în care oricare 2 $\mu o$ sunt compatibile între ele

• **CLASĂ DE COMPATIBILITATE MAXIMĂ : MCC**

Este acea clasă de compatibilitate la care nu se mai poate adăuga nicio microinstrucțiune fără a pierde compatibilitatea.

$Mcc(\mu o) = \{\mu o_j \mid \forall \mu o_i \notin Mcc(\mu o), \exists \mu o_j \in Mcc(\mu o)$ a.î. $\mu o_i \in \mu ic_K$ și $\mu o_j \in \mu ic_K\}$

• **COST CLASĂ DE COMPATIBILITATE**

- este nr. biți necesari pentru a codifica toate $\mu o$ din acea clasă

$$cost(CC(\mu o)) = \lceil \log_2(|CC(\mu o)| + 1) \rceil$$

↑ pentru NOP

$$Cost(\text{Structură } \mu ic) = \sum Cost(CC_i(\mu o))$$

• **CLASA DE INCOMPATIBILITATE : IC**

Două $\mu o$ sunt incompatibile dacă $\exists$ cel puțin o $\mu ic$ a.î. $\mu o_i \in \mu ic$ și $\mu o_j \in \mu ic$.

• **CLASA DE COMPATIBILITATE MAXIMĂ ASOCIATĂ UNEI MIC : (AMCC asociată lui MIC)**

Pt. $\forall$ IC, clasele de compatibilitate maximă ce conțin un element din ic se numesc AMCC.

$$AMCC = \{MCC \mid \forall \mu o \in MIC_m, \exists \mu o \in MCC \text{ și } \mu o \in \mu IC_m\}$$

Proprietăți:

- Pt. $\forall$ MIC, reuniunea MCC asociate acoperă întreg blocul de $\mu_o$.
- Pt $\forall$ MIC, reuniunea a K MCC, $K < |MCC|$, nu acoperă întreg blocul de $\mu_o$
- O partiție a setului de $\mu_o$ în $g+h$ câmpuri are cost mai mare decât partiția cu $g$ câmpuri

**9. Algoritm euristic + optim de impartire pe niveluri**

PAS 1

$\mu PT = \emptyset$

PAS 2

Definim setul de $\mu_o D$ și $\mu_o ND$

$\mu_o D = \{\mu_o \mid \mu_o \in \mu_B(\mu_o)$ și $\nexists\ \mu_{o_i} < \mu_o$ să nu fi fost într-o $\mu ic\}$

$\mu_o ND = \{\mu_o \mid \mu_o \in \mu_B(\mu_o), \mu_B(\mu_o) \setminus \mu_o D\}$ la nivelul $K\}$

PAS 3

Din $\mu_o D$ generăm $\mu ic$

$\{\mu ic\} = \{\mu ic_j, \dots, \mu ic_K\}$

Se alege $\mu ic$ cu nr. maxim succesori $\Rightarrow \mu ic_h$

$\mu PT = \mu PT \cup \mu ic_h$

$\mu_o D = \mu_o D \setminus \mu ic_h \cup \{\mu_{od}\}_h$

$\mu_o ND = \mu_o ND \setminus \{\mu_{od}\}_h$

PAS 4

Dacă $\mu_o ND \neq \emptyset$, atunci salt la 3

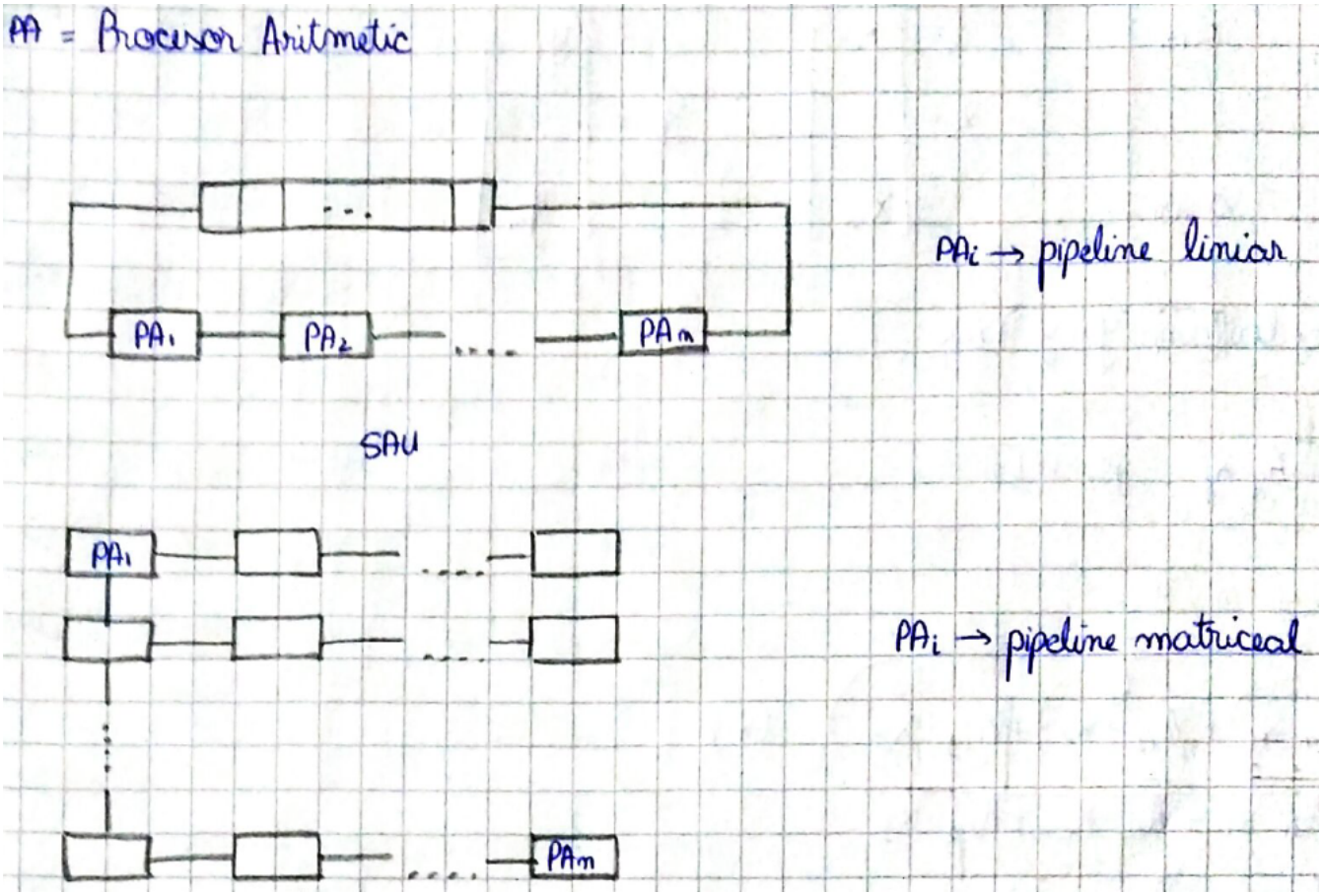
Altfel dacă $\mu_o D \neq \emptyset$ atunci salt la 3

altfel $\mu PT$ finală

**10. Dati exemple de sisteme cu arhitectura paralela**

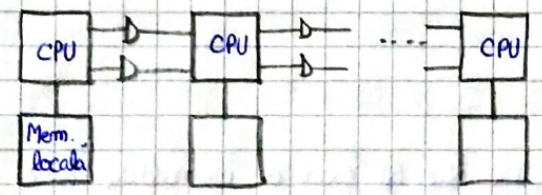
IBM's Blue Gene/P massively parallel supercomputer.
Blue Gene/L - eServer BlueGene Solution IBM
Roadrunner - BladeCenter Cluster IBM
Ranger - SunBlade - opteron quad 2Ghz infiniband- Sun Microsystems
Jaguar-Cray XT4 QuadCore 2.1Ghz - CrayInc.

**11. Modele sistolic (memorie partajata cu transfer de mesaje)**



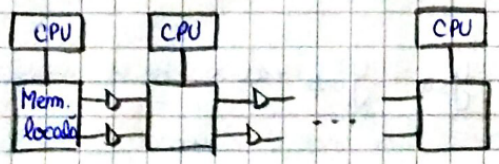
METODE DE COMUNICARE ÎN SISTEMELE SISTOLICE:

(1)




După prelucrare, punem într-un buffer, iar următorul știe să preia de acolo

(2)

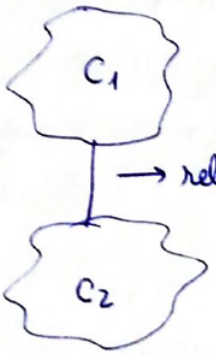



Prelucrez eu și pun rezultatul în memorie. În acest moment există un acces direct de memorie care mută datele de ieșire în altă zonă de memorie ca fiind date de intrare.

⇒ necesită implementarea unui DMA în sistem

**12. Concatenarea sistemelor de sarcini**

Două sisteme de sarcini $C_1$ și $C_2$ pot fi concatenate ($C=(C_1,C_2)$), graful asociat obținându-se prin introducerea unui arc de la nodul terminal al lui $C_1$ la nodul inițial al lui $C_2$.




→ relație precedență : $\left.\begin{matrix} S_i \in C_1 \\ S_j \in C_2 \end{matrix}\right\} \Rightarrow S_i \prec S_j$

$C = (C_1, C_2)$

$C_1 = (S_1, \alpha_1)$ unde $\alpha_1 = a_1 \dots a_{2n}$ și $S_1 = \{S_{11}, \dots, S_{1n}\}$

$C_2 = (S_2, \alpha_2)$ unde $\alpha_2 = b_1 \dots b_{2m}$ și $S_2 = \{S_{21}, \dots, S_{2m}\}$

$\Rightarrow \alpha = a_1 \dots a_{2n}\, b_1 \dots b_{2m}$

**13. Sisteme echivalente**

Două sisteme cu aceeași mulțime de sarcini sunt echivalente dacă sunt determinate și dacă pentru aceeași stare inițială produc aceeași secvență de valori.

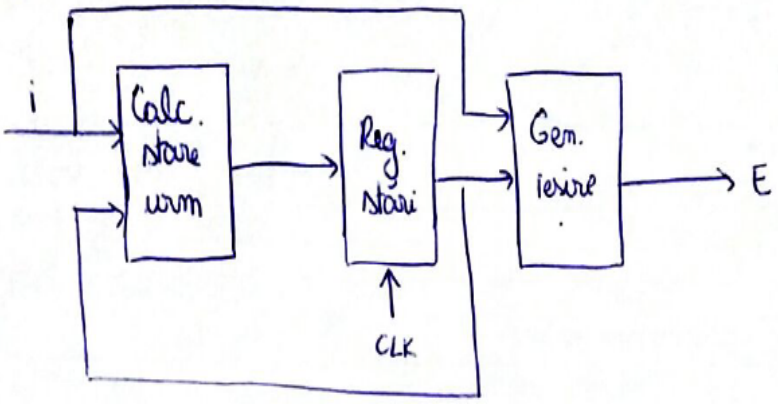
$C' = (S, \alpha')$

$C = (S, \alpha)$

**14. Structura unei unitati de comanda microprogramate**

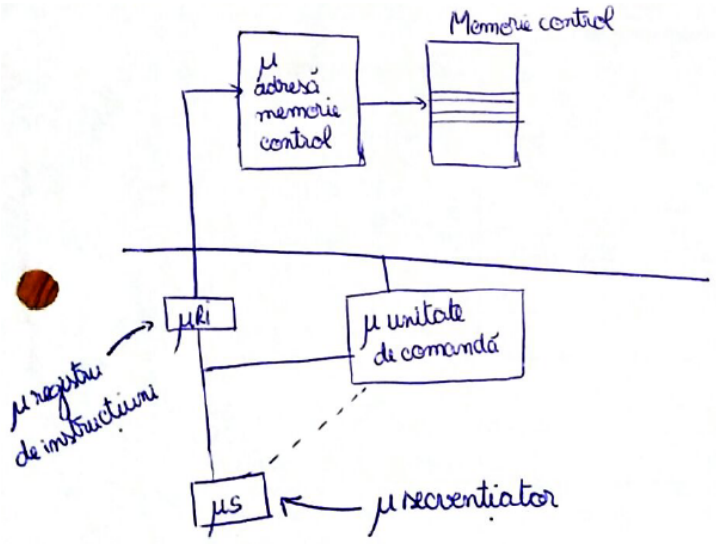
Poate fi implementată în 2 moduri:

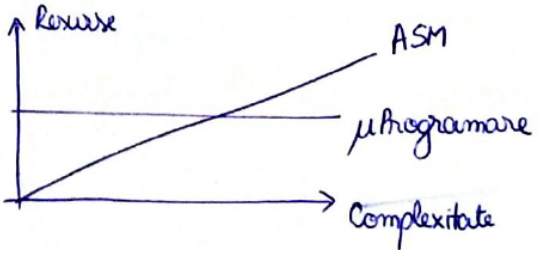
- Algorithm State Machine




- Microprogramare:

Stările sunt reprezentate ca celule de memorie, iar trecerea de la o stare la alta este făcută de o unitate mică de comandă.




**15. Indicatori de performanta (prelucrari paralele)**

• RATA DE EXECUȚIE: măsoară rata de obținere a unor rezultate în unitatea de timp

Unități de măsură →

- MIPS (milioane instrucțiuni pe secundă)
  - ↳ inadecvat pt. o mașină SIMD care execută aceeași instrucțiune pe mai multe fluxuri date
- MOPS (milioane operații pe secundă)
  - ↳ nu ține seama de natura operațiilor sau lungimea cuvintelor
- MFLOPS (milioane de operații în virgulă mobilă pe secundă)
  - ↳ adecvată numai pentru aplicații numerice
- MLIPS (milioane de inferențe logice pe secundă)
  - ↳ adecvată pentru aplicații de IA

- **VITEZA DE PRELUCRARE**

$V_p = \frac{T_1}{T_p}$, unde $\begin{cases} T_1 = \text{timpul necesar pt. prelucrare cu 1 procesor} \\ T_p = \text{—11—} \quad \text{p procesoare} \end{cases}$

↑ raportul dintre prelucrarea secvențială și cea paralelă
scoate în evidență creșterea în viteză datorită paralelismului

$V_p \geq 1$

Ideal : $T_1 = p \cdot T_p$

- **EFICIENȚA** : raportul dintre viteza de prelucrare și număr procesoare

$E_p = \frac{V_p}{p} = \frac{T_1}{p \cdot T_p} < 1$

- **REDUNDANȚA**

$R_p = \frac{Q_p}{Q_1}$ unde $Q$ = nr. operații efectuate în timp

↑ reflectă timpul pierdut cu overhead-ul

- **UTILIZARE**

$U_p = \frac{Q_p}{p \cdot T_p} \leq 1$

↑ raportul dintre numărul de operații necesare efectuării calculelui cu p procesoare și numărul de operații care ar fi putut fi efectuate cu p procesoare în timpul $T_p$

**16. Diferenta intre sistemele microprogramate si microprogramabile**

Sisteme microprogramate → modalitatea de implementare a UC, fără a oferi resurse hardware și suportul de programe pt. accesul utilizatorului la nivelul μprogramului

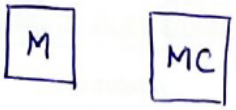
Sisteme microprogramabile → oferă atât resurse hardware, cât și facilitățile software pentru accesul utilizatorului la nivelul μinstrucțiunii

**17. Modalitati de implementare a memoriei de control**

Poate fi privită din 2 perspective → relația dintre memoria de control (MC) și memoria principală (M)
→ Structura MC

Perspectiva 1 : Relația dintre MC și M

- MC separată de M, atât din punct de vedere logic, cât și fizic

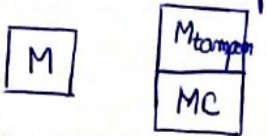



$V_{MC} \gg V_M$

- MC implementată în același spațiu fizic și de adresare cu M

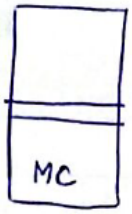
Memoria trebuie să fie suficient de rapidă ca să poată fi folosită ca MC și destul de ieftină ca să fie folosită ca M.

- MC este implementată separat de M, dar este încărcată din aceasta prin intermediul unei memorii tampon.



## Perspectiva 2 : Structura MC

- MC cu o μinstrucțiune pe cuvânt




→ un cuvânt din memorie ⇒ o μinstrucțiune

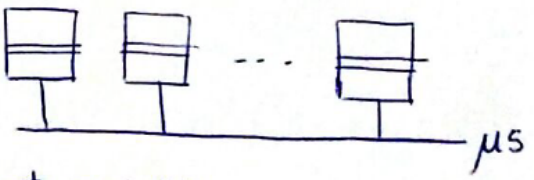
- citire μinstrucțiune ⇒ un singur acces la MC

- MC cu organizare pe pagini

Pt. un grup de μinstrucțiuni se face o singură citire, iar secvențiatorul se deplasează la fiecare din ele

Dezavantaje:

- microinstrucțiuni mai lungi
- organizarea în memorie de către compilator este mai grea




- MC cu organizare pe blocuri

Există 2 tipuri de adrese → adrese de μinstrucțiuni din același bloc cu μinstrucțiunea curentă
→ adrese de blocuri

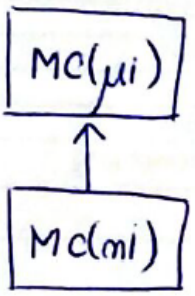
- micșorarea lungimii instrucțiunii
- timp suplimentar cu comutarea adreselor de blocuri

- MC divizată

- Se folosesc 2 unități de memorie → memorie de μinstrucțiuni (Mi) ⇒ păstrează toate μi distincte
→ memorie de adrese de μinstrucțiuni (MA) ⇒ păstrează programul prin reținerea adreselor de μi

- lungime cuvânt$_{MA}$ << lungime cuvânt Mi
- pentru execuția unei μi sunt necesare 2 adresări : una la MA și una la MI

- MC structurată pe 2 niveluri
  - flexibilitate mare
  - o instrucțiune mașină este interpretată de un set de μi din MC(μi). La rândul ei, fiecare μi este interpretată de o secvență de mi din MC(mi)




**18. Organizarea microoperatiilor**

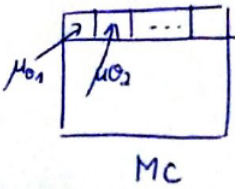
Trebuie să ținem cont de următoarele lucruri:

- gradul de paralelism pe care vrem să îl atingem
- structura mașinii de bază
- gradul de optimizare a lungimii cuvântului de control

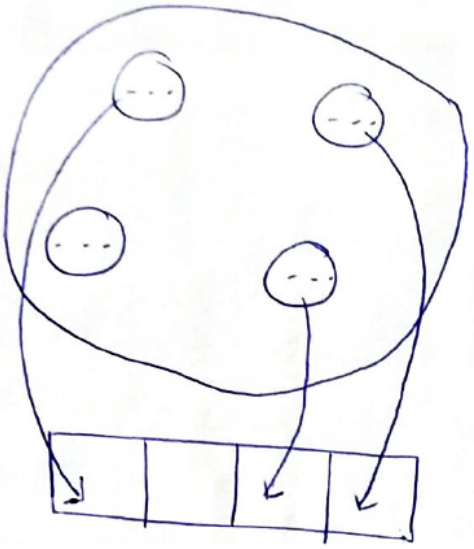
Există 6 tipuri de implementări:

- CONIFICARE VERTICALĂ / MAXIMALĂ
  - fiecare microinstrucțiune specifică o microoperație
  - se rulează secvențial
  - cuvânt de control are lungimea minimă în acest caz: $\lceil \log_2 (n+1) \rceil$
    ↑ avantaj
  - Dezavantaje:
    - eliminarea controlului paralel asupra resurselor
    - lipsă de flexibilitate în introducerea de noi microoperații

- **CODIFICARE ORIZONTALĂ**
  - fiecare μoperație este pusă în corespondență cu un bit din cadrul cuvântului de control
  - avantaj: asigură paralelism maxim și are o flexibilitate mare
  - dezavantaj: lungime mare a memoriei de control



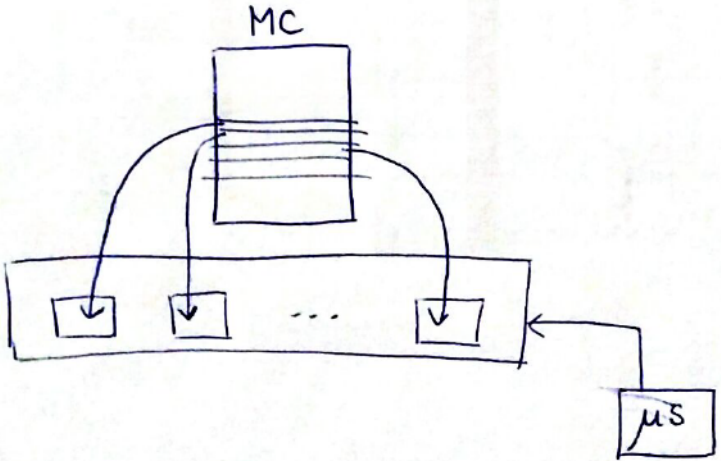

- **CODIFICARE MINIMALĂ / OPTIMĂ**
  - combină flexibilitatea și paralelismul de la codificarea orizontală cu eficiența codificării verticale
  - se stabilesc operațiile care nu se pot executa în paralel.
  - grupările cu operațiile care se exclud reciproc ⇒ codificare verticală + le punem în același câmp
  - grupările cu operațiile care nu se exclud reciproc ⇒ codificare orizontală + le punem în câmpuri diferite



- Este necesar un decodificator pentru fiecare câmp

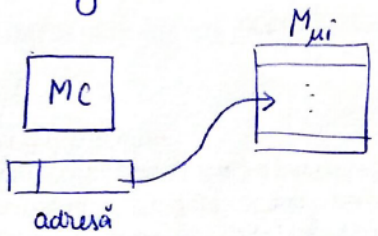
• CODIFICARE CU CONTROL REZIDUAL

- cuvântul de control nu controlează direct resursele, ci prin intermediul reg. de control rezidual
- instrucțiunile pot să modifice valoarea unuia sau a mai multor registre
- avantaj: asigură o economie a MC atunci când se realizează aceeași operație în mod repetat




• CODIFICARE CU CONTROL PRIN ADRESE

- formă simplificată a conceptului de nanoprogramare
- fiecare μi e memorată o singură dată
- dezavantaj: necesită 2 acces la memorie în cadrul unui ciclu de microinstrucțiune
- avantaj: economie de memorie




• CODIFICARE MIXTĂ

- microinstrucțiunea este împărțită în câmpuri
- putem avea în fiecare câmp → microoperații
  → câmpuri
  → control al câmpurilor
  → adresă

**19. Legatura dintre algoritmi paraleli si arhitecturi paralele**

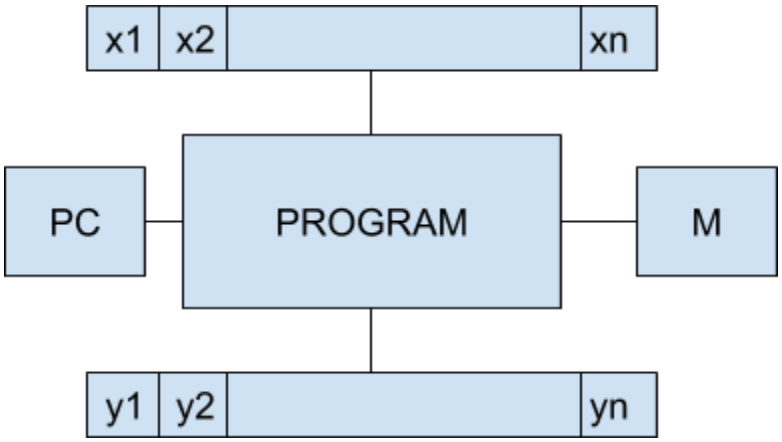
| Algoritm paralel | Arhitectură paralelă |
|---|---|
| • Granularitate de prelucrare<br>Unde mă opresc? Job/Task/proces? | • Complexitate procesor |
| • Control concurent<br>Se referă la schema de selecție a modulelor pentru execuție care trebuie să satisfacă dependență de date și de control | • Mod de operare |
| • Structurarea datelor<br>Structuri date, mecanisme de acces la date, etc. | • Structurarea memoriei |
| • Geometria comunicației<br>Se referă la șablonul de interacțiune între module | • Rețele de comutare<br>Ex: CrossBar, Delta |
| • Complexitate algoritm | • Număr procesoare, dimensiune memorie et |

## I. d

## Modele de calcul paralel

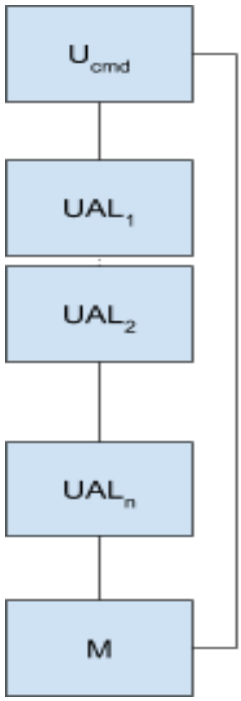
A. RAM (Random Access Machine)

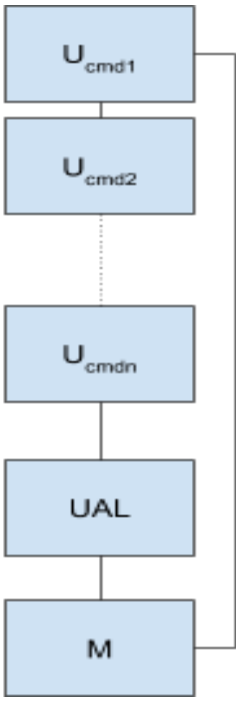
- Masina de baza monoprocesor
- Similar cu modelul Turing
- Analiza complex algoritmilor si dezvoltarea lor
- Punctul de plecare pentru modelele paralele
- La fiecare moment de timp se executa cate o instructiune



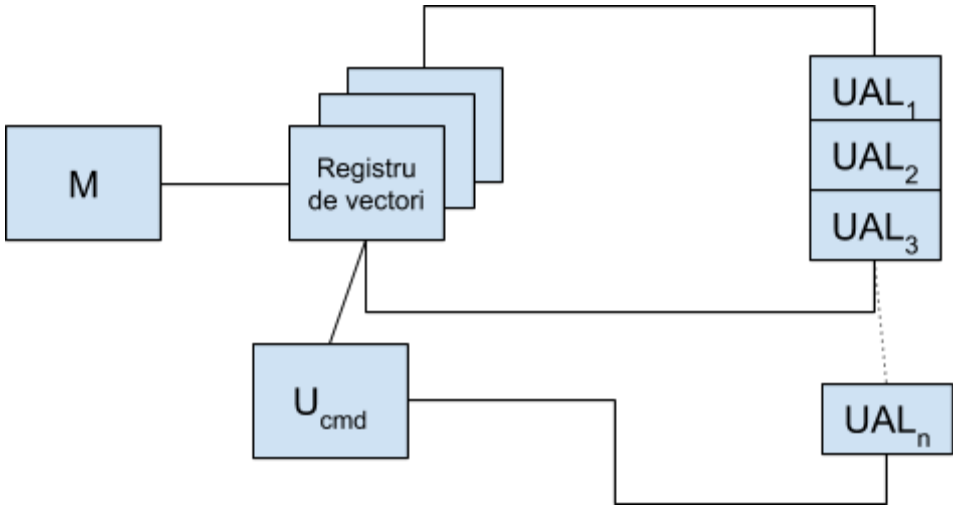

B. Pipeline

- La nivel unitate de comanda / la nivel unitate de prelucrare
- Imparte un task T in subtaskuri $T_1$...$T_n$. Fiecare subtask e atribuit unui element de procesare.

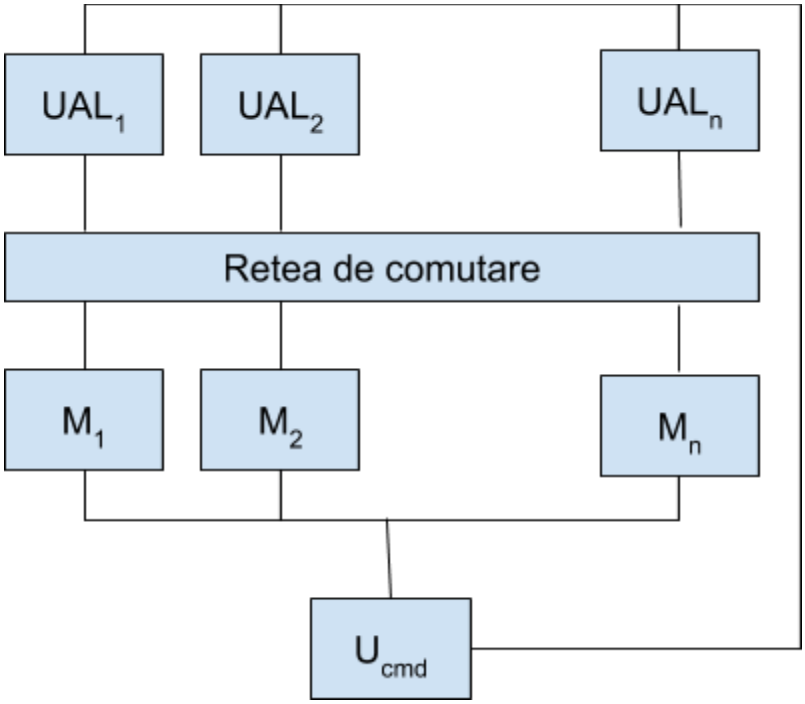


C. Procesare de vectori
   - Set de instructiuni care trateaza vectorul ca un operand simplu
   - Sisteme monoprocesor care au ca extensie procesarea de vectori
   - SIMD

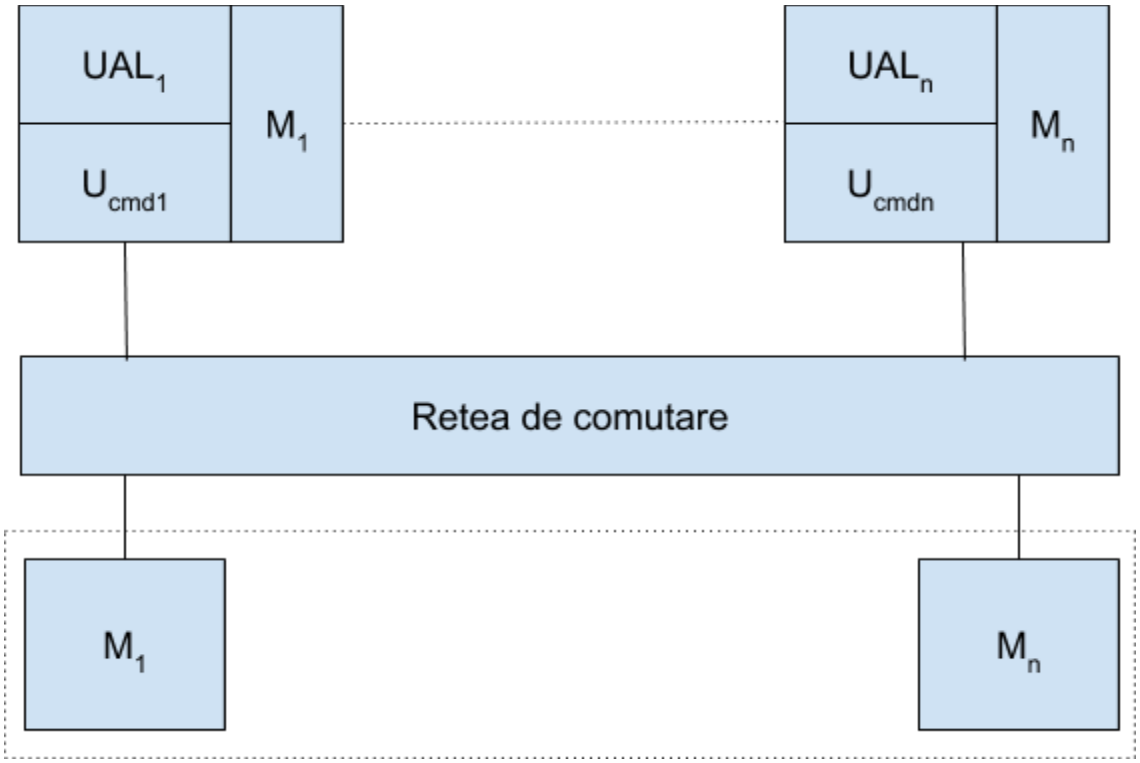



D. Procesare de “masive de date” (MPP)
   - Calculatoare care constau din unitati aritmetice multiple supervizate de o singura unitate de control, care efectueaza aceleasi op la un mom dat
   - Asemanator cu SIMD + UMA



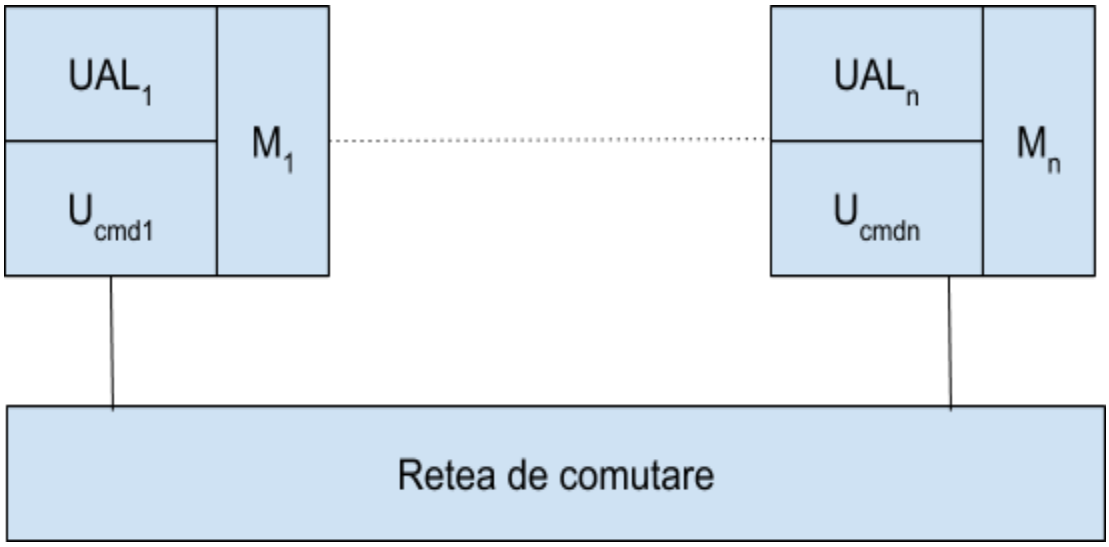
E. Multiprocesoare cu memorie partajata
   - Fiecare procesor are propriile UAL, Ucmd si M
   - Comunicarea se face prin intermediul unei retele de comutare, cu module de memorie partajata
   - Reteaua de comutare permite atat schimbul de informatii cat si accesul la fiecare modul al memoriei partajate
   - MIMD + NUMA (acces neuniform la mem. Partajată vs mem. locală)

..




F. Modelul PRAM (Parallel Random Access Machine)
   - Programele sunt diferite, comunicarea intre P si M este considerata ca se face  sincron, dar secvential daca se face la acelasi submodul de memorie
   - Memoria comuna e disponibila tuturor proceselor
   - 4 posib de acces la memorie: EREW, ERCW, CREW, CRCW

G. Multiprocesor cu transfer de mesaje
   - Fiecare procesor dispune de elemente de procesare
   - Interactiunea se face prin mesaje, nu acces direct




H. Procesarea sistolica
   - Elemente de procesare identice, asezate in pipeline liniar/matriceal
   - Utilizat pentru operaţii pe vectori, matrice
   - Comunicarea între PA se face: la nivel de CPU (comunicare directă între CPU-uri) sau prin DMA (comunicare indirectă între CPU-uri, prin DMA)

I. Modelul data flow
   - Specifica efectul unor operatii imediat ce operanzii sunt disponibili -> elimina necesitatea existentei contorulului de program
   - pentru a realiza aceasta, fiecarui operand i se asociaza un token prin care se specifica daca e disponibil pentru prelucrare
   - Nu necesita adrese pt memorare
   - Nu se execută instr., ci se asociază fiecărei date o stare de disponibilitate -> execuţie asincronă ce începe când datele sunt disponibile
   - Viteză ridicată & potenţial ridicat de paralelism
   - Uşor de integrat dpdv al sincronizării & comunicării
   - Pot exista timpi mari de aşteptare, overhead mare de control, iar prelucrarea structurilor de date (vectori, matrice etc) e foarte dificilă

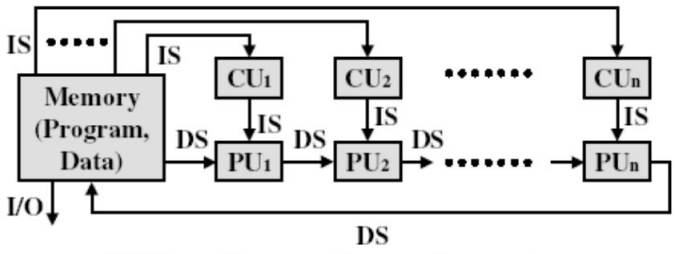
## II. Subiecte examen

1. Modele de calcul paralel. Comparatie data flow-transmitere de mesaje(3)
    - La data flow nu sunt necesare adrese pentru memorare, la transmitere de mesaje fiecare procesor are elementele proprii de procesare(UAL, $U_{cmd}$, M)
    - La data flow comunicatia si sincronizarea sunt usor de integrat, in schimb la transmiterea de mesaje au un cost ridicat din cauza timpului in plus necesar trimiterii de mesaje
    - Ambele au un potential ridicat de paralelism

2. Modele de calcul paralel. Comparatie intre modelul cu memorie partajata si modelul sistolic.
    - La modelul sistolic unitatile de procesare sunt asezate in pipeline(liniar sau matriceal), in schimb la cel cu memorie partajata fiecare proces are propriile unitati de procesare(UAL, Ucmd,M) si de asemenea au si o memorie comuna
    - Timpul de acces la memoria comuna in cazul memoriei partajate este mai mare deoarece mai multe procese pot incerca simultan sa o acceseze, dar doar unul poate intr-un ciclu de ceas. La pipeline este impartita in mai multe module, astfel se pot accesa mai multe module simultan.
    - Sistolic - MISD, partajata: MIMD

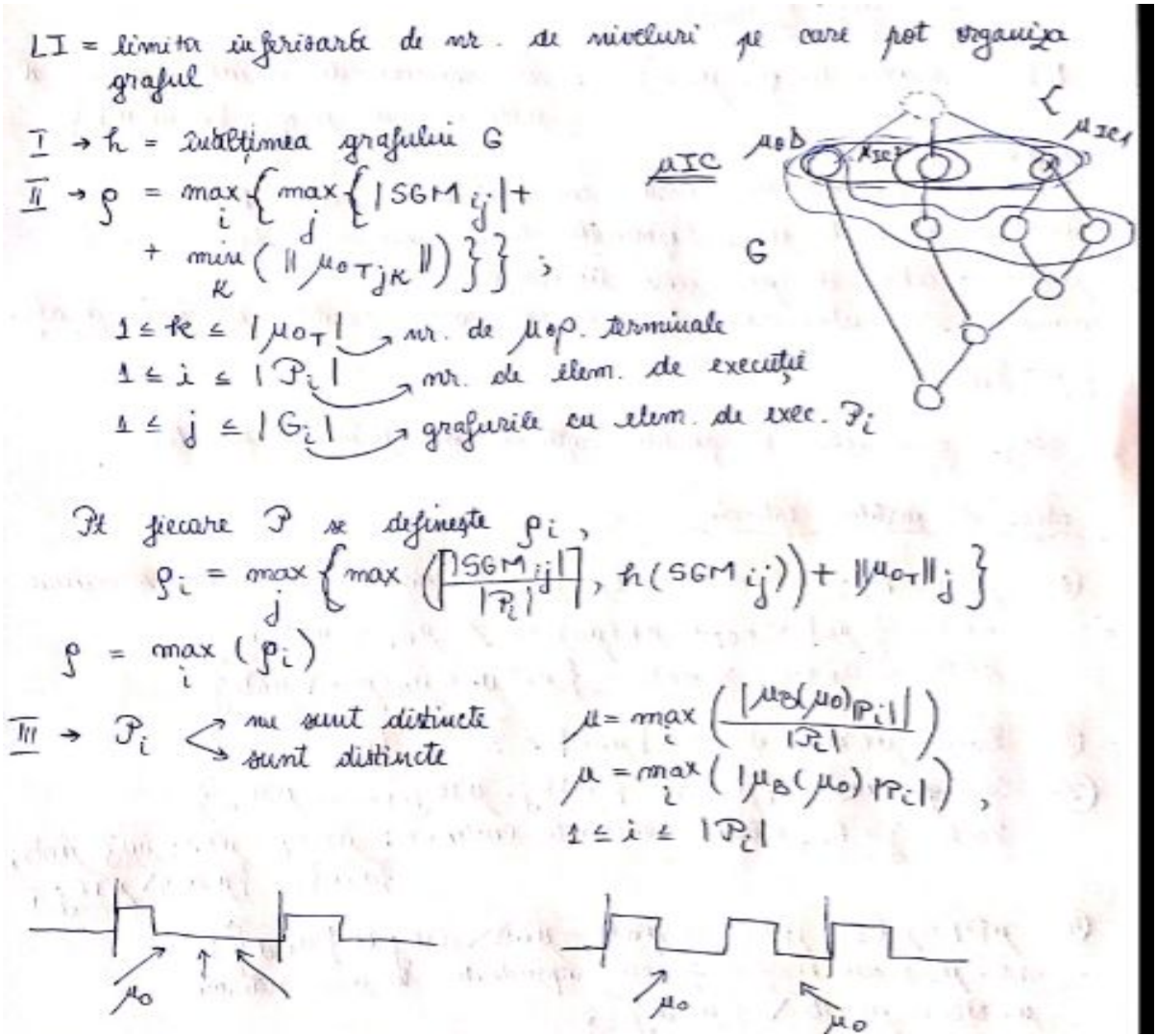
## MISD (Multiple Instruction Streams Over A Single Data Streams)

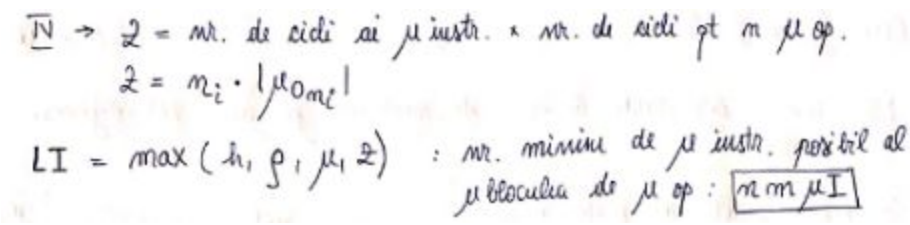
- ❖MISD
  - ✓Processor arrays, systolic arrays
  - ✓Special purpose computations




MISD architecture (the systolic array)

3. **Limita inferioara** (4) - cred ca asta vrea… dar este extrem de mult de scris...





4. Gustafson

Gustafson

w - workflow

$\alpha$ - procent secvential

1- $\alpha$ - procent paralel

w'= $\alpha w + (1-\alpha)*n*w$

$T_1 = \alpha w + (1-\alpha)*n*w$

$T_n = \alpha w + (1-\alpha)*w$

$$S_n = \frac{\alpha w + (1-\alpha)*n*w}{\alpha w + (1-\alpha)*w}$$

4'. Sun-Ni

**Sun-Ni's Law** (or Sun and Ni's Law), is a memory-bounded speedup model which states that as computing power increases the corresponding increase in problem size is constrained by the system's memory capacity.

w* = the scaled workload under a memory space constraint

w - workload-ul normal

$\alpha$ - procent secvential

G(m) - the function that reflects the parallel workload increase factor as the memory capacity increases m times

w* = $\alpha w + (1 - \alpha)\, G(n)\, w$

$$S^{**} = \frac{\alpha w + (1-\alpha)\, G(n)\, w}{\alpha w + \frac{(1-\alpha)\, G(n)\, w}{n}} = \frac{\alpha + (1-\alpha)\, G(n)}{\alpha + \frac{(1-\alpha)\, G(n)}{n}}$$

**Pentru subiecte de comparatie:** Amdahl's law states that the sequential portion of the problem (algorithm) limits the total speedup that can be achieved as system resources increase. Gustafson's law suggests that it is beneficial to build a large-scale parallel system as the speedup can grow linearly with the system size if the problem size is scaled up to maintain a fixed execution time.[4] Yet as memory access latency often becomes the dominant factor in an application's execution time,[5] applications may not scale up to meet the time bound constraint.[1][2] Sun-Ni's Law, instead of constraining the problem size by time, constrains the problem by the memory capacity of the system, or in other words bounds based on memory. Sun-Ni's Law is a generalization of Amdahl's Law and Gustafson's Law. When the memory-bounded function *G(M)=1*, it resolves to Amdahl's law, when the memory-bounded function *G(M)=m*,the number of processors, it resolves to Gustafson's Law.

5. Amdahl vs Worlton

Amdahl - 2k18: Formula prin care se stabileste o limita a cresterii de viteza in structurile paralele in raport cu cele secventiale

Worlton - tcare aproximeaza operarea unui multiprocesor

Worlton

- $t_s$ - timpul de sincronizare
- $t_o$ - timpul de overhead
- t - timpul mediu de executie al unui task
- p - nr de procesoare
- N - nr de taskuri
- $T_1$ - N*t
- $T_{Np}$ - $t_s$ + (N/p)*(t+$t_o$)

$$VNp = \frac{T1}{Tnp} = \frac{1}{\frac{ts}{Nt}+\frac{1}{N}*[\frac{N}{p}](1+\frac{t0}{t})}$$

$\frac{ts}{Nt}$ - reducerea timpului de sincronizare + marirea timpului intre sincronizari (efectul sincronizarii)

$\frac{N}{p}$ - cresterea numarului de procesoare, nr de taskuri multiplul nr de procesoare

$\frac{t0}{t}$ - cresterea lui t(granularitatea taskurilor) - efectul de overhead

6. Legea lui Amdahl

$R_H$ - rata de executie pe p procesoare

$R_L$ - rata de executie pe 1 procesor

f - procentul de paralelism

1-f - procentul de secventialitate

$$R(f) = \frac{1}{\frac{f}{Rh}+\frac{1-f}{Rl}}$$

$$R(f) = \frac{1}{(1-f)+\frac{f}{N}}$$ N- nr de procesoare.

7. **Compatibilitate si incompatibilitate(2)** - sunt si in cursurile de andrei.cisco - dar trebuie descifrate(cursul 9 Pag 1), dar o sa o fac mai tarziu - acolo imi par foarte vag enuntate.

**Def. 1**

Fie $mPt=\{mIC_1,mIC_2,...,mIC_{[mPt]}\}$ partiția unui microsubbloc în microinstrucțiuni complete și $MB(mO)=\{mO_1,mO_2,..., mO_{[MB]}\}$ setul de microoperații distincte din cadrul microsubblocului.

Două microoperații $mO_i$ și $mO_j$ sunt **compatibile** dacă pentru orice k, $1 \leq k \leq [mPT]$, dacă $mO_i \in mIC_k$ atunci $mO_j \notin mIC_k$.

Compatibilitatea între două microoperații trebuie privită în sensul că cele două microoperații nu sunt specificate (nu sunt active) niciodată împreună în cadrul unei microinstrucțiuni din microbloc. Controlul resurselor sistemului microprogramat nu necesită niciodată efectuarea în paralel a celor două microoperații.

**Def. 2**

Două microoperații $mO_i \in MB(mO)$, $mO_j \in MB(mO)$ sunt **incompatibile** dacă există cel puțin o microinstrucțiune completă mIC astfel încât $mO_i \in IC_k$ și $mO_j \in mIC_k$.

8. Clase de compatibilitate, incompatibilitate - aici nu gasesc incompatibilitate - la fel ca mai sus se gasesc in cursul 9

O clasă de compatibilitate CC(mO) este un set (subset) al mulțimii MB(mO) în care oricare două microoperații sunt compatibile între ele.

$CC(mO)=\{mO \mid$ pt orice $mO_i, mO_j \in CC(mO)$ avem $mO_i$ compatibilă cu $mO_j\}$

- o singura definitie… nici nu ai ce scrie la ea

9. **Costul clasei de compatibilitate - la fel se gaseste ca mai sus si asta se gaseste in cursul 9 pag1(jos)-2 - o sa o scriu mai tarziu si pe asta din cursul 9**

Costul de implementare a unei clase de compatibilitate (măsurat în numărul de biți necesari pentru codificare) este dat de implementarea codificării verticale a microoperațiilor ce compun clasa.

$$\text{Cost } CC_i = [\log_2(|CC_i|+1)]$$

iar costul total de implementare al cuvântului de control

$$\text{Cost } \sum_{i=1}^{k} CC = S\ [\log_2(|CC_i|+1)]$$

unde k este numărul de clase de compatibilitate.

**10. Sisteme microprogramate si microprogramabile(3)**

Sistemele microprogramate se refera la modalitatea de implementare a unitatii de comanda fara a oferi resursele hardware si suportul de programe pentru accesul utilizatorului la nivelul programului.

Sistemele microprogramabile ofera atat resursele hardware cat si facilitatile software pentru accesul la nivelul microinstructiunilor

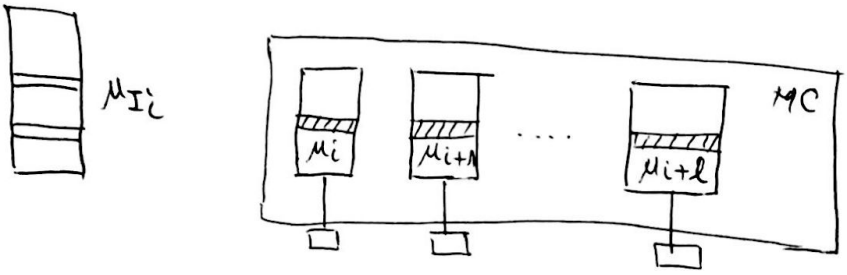
**11. Organizarea memoriei de control**

Org mem de control in functie de mem principala:

1. M si Mc sunt entitati separate
2. M si Mc sunt aceeasi entitate
3. M si Mc sunt entitati separate, dar sunt incarcate in aceeasi entitate

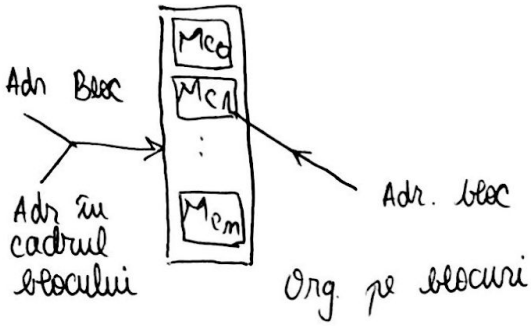
Org mem de control:

1. Pagini
2. Blocuri
3. 2 nivele

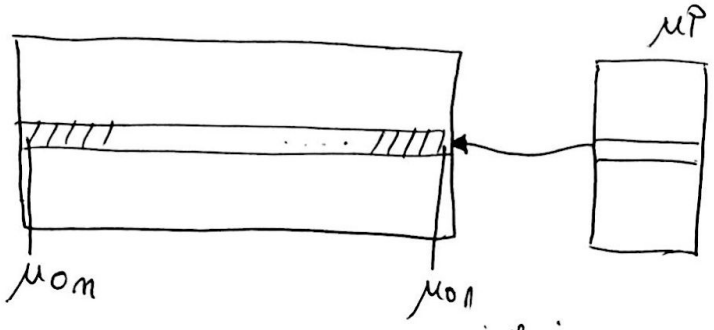



- citesc 1 instr.
- citesc și execut

Org. pe pagini




Org. pe blocuri




Org. pe 2 niveluri

- reducere foarte mare de spațiu de mem.
- de 2 x timpul de acces la mem.

-

**12. Algoritmi de partitionare** - vezi urmatoarele 2 exercitii

**13. Algoritmul de partitionare (in 8 pasi)**

Alg. de partiție optimă

① $\mu PT = \emptyset$ ; $\mu PTA = \mu_B(\mu_0)$; se consideră că se exec. secvențial

$\mu_0 D = \{\mu_0 \mid \forall \mu_{0i} \in \mu_B(\mu_0) \Rightarrow \nexists\ \mu_{0i} < \mu_0\}$ ;

$\mu_0 ND = \mu_B(\mu_0) \setminus \mu_0 D = \{\mu_0 \mid \mu_0 \in \mu_B(\mu_0) \setminus \mu_0 D\}$

② Dacă $\mu_0 ND = \emptyset$ și $|\mu_0 D| \leq 3 \Rightarrow$ ⑤

③ Se generează $\{\mu IC\} = \{\mu IC_j, \mu IC_{j+1}, \dots, \mu IC_k\}$

Dacă $j \neq k$, atunci salvează CONTEXT : $\mu PT_j = \mu PT$; $\mu_0 j = \mu_0 D$;

$\{\mu IC\}_j = \{\mu IC\} \setminus \mu IC_j$ ;

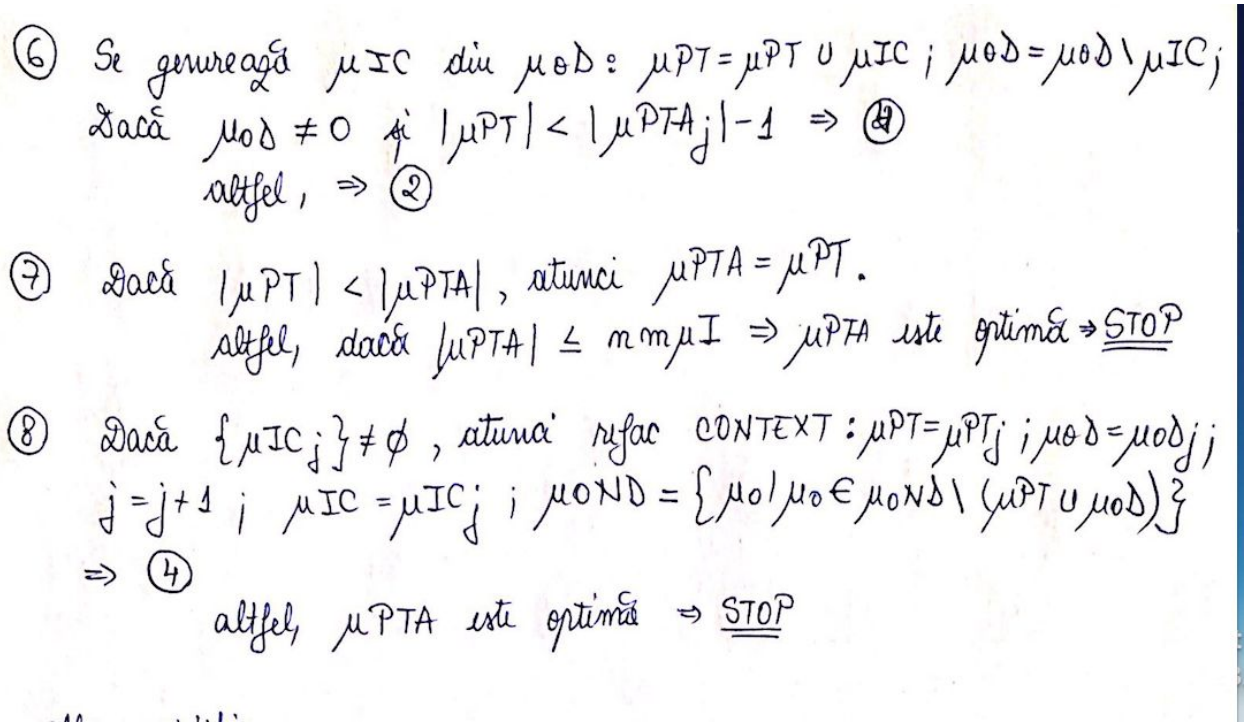
④ $\mu PT = \mu PT \cup \mu IC_j$ ; $\mu_0 D = \mu_0 D \setminus \mu IC_j \cup \{\mu_0 d\}^*$;

obs: $\mu_0 d$ nu conține și cele dependente de $\mu_0 d$ anterior

$\mu_0 ND = \mu_0 ND \setminus \{\mu_0 d\}^*$;

Dacă $|\mu_0 D| \neq 0$ și $|\mu PT| \leq |\mu PT_j| - 1$, atunci ⇒ ②

⑤ Dacă $\mu_0 D = \emptyset$, atunci ⇒ ⑦




14. **Algoritm de partitionare a ul folosind uO** - asta pare cea mai aproape ca foloseste microoperatii

// Algoritm euristic

1. Pas 1. Initializare

$$\begin{cases} \mu PT \neq \Phi \text{ , } \mu OD = \{\mu o \mid \mu o \in \mu OS \neq \mu o' < \mu o\} \\ \mu OD = \mu B(\mu o) \setminus \mu OD \end{cases}$$

$p_{succ}(\mu o)$ = nr de succesori ai operatorului respectiv

$p_{succ}(\mu JC) = \sum_{\mu o \in \mu JC} p_{succ}(\mu o)$

2) daca $\mu OMS$ atunci pas 4

3) genereaza setul $\{\mu JC\} = \{\mu JC_1, \mu JC_{j+1}, \ldots \mu JC_k \ldots$ ordonat.

- consideram sortata aceasta multime dupa numarul de succesori ai $\mu JC$
- $p_{succ}(\mu JC_j) \geq p_{succ}(\mu JC_{j+1}) \geq \ldots \geq p_{succ}(\mu JC_k)$

$\mu PT = \mu PT \cup \mu JC_j$

$\mu OS = \mu OS \setminus \mu JC_j \setminus \{\mu od\}_j$

$\mu ONS = \mu ONS \setminus \{\mu od\}_j$

salt 2

4. Genereaza $\{\mu JC\} = \{\mu JC_h \ldots \mu JC_k\}$ le ordonam dupa un criteriu

$\mu PT = \mu PT \cup \mu JC_h$

$\mu OS = \mu OS \setminus \mu JC_h$

// salt pas 2 - comentat

daca $|\mu OS| \neq 0$ atunci salt 4

altfel $\mu PT$ este cea aproape optima.

**15. Teorema de suficienta(5)**

**Teorema de suficienta**

Sistemul de sarcini format din sarcini mutual neinterferente este determinat.

********** **daca cere si:**

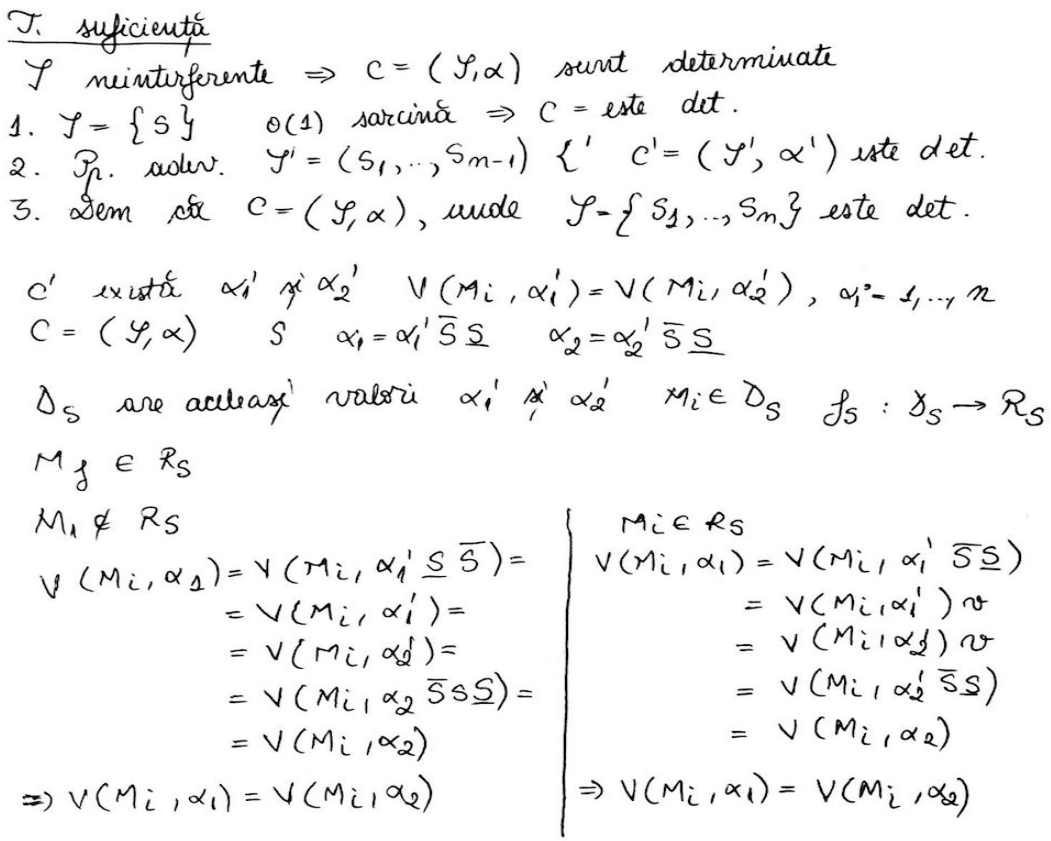
**Teorema de necesitate:**

Fie dat un sistem de sarcini neinterpretat, dar cu DS si RS≠0, sistemul C este determinat pt ∀ interpretare a sarcinilor sale ⇔ sarcinile sunt neinterferente

**Sisteme echivalente de sarcini:**

Doua sisteme sunt echivalente ⇔ sunt determinate si produc aceleasi secvente de valori pentru o stare initiala data

**=**




**16. Indicatori paraleli(3)**

a. Rata de executie - producerea unui rezultat in unitatea de timp

i. MIPS - milioane de instructiuni/sec
ii. MOPS - milioane de operatii/sec
iii. MFLOPS - milioane de op in virgula mobila/sec
iv. LIPS(MLIPS, GLIPS) - milioane de inferente logice/sec

b. Viteza de prelucrare

Vp = T1/Tp >> 1 (t 1 proc/ t p proc) -> raport intre prelucrarea paralela si cea secventiala

p*Tp >> T1 -> se consuma timp cu sincronizarea, comunicarea, overhead creat de interactiunea intre procese

Caz ideal p*Tp = T1

c. Eficienta de prelucrare

Ep = Vp/p = T1/(p*Tp) < 1

d. Redundanta

Rp = Op/O1 (nr op p proc/nr op 1 proc) -> t pierdut de overhead

e. Utilizarea

Up = Op/(p*Tp) < 1

**17. Sistem de sarcini determinat. Secvente de executie. Secventa partiala de executie(2)**

a. Un sistem de sarcini determinat este acel sistem care se executa in paralel si care coopereaza intre sarcini pentru implementarea functiilor, care produce acelasi rezultat indiferent de rata de executie a unei sarcini independente si indiferent de ordinea in care se executa sarcinile paralele.

b. Secvente de executie

$\alpha = \alpha_1\alpha_2...\alpha_{2n}$ $\alpha_i = \bar{S}$ sau $a_i = \underline{S}$

$\bar{S}$ sau $\underline{S}$ apar o data in $\alpha$

1. $a_j = \bar{S}_i$ $a_k = \underline{S}_j$ => j < k (initierea e intotdeauna inaintea terminarii)
2. $a_l = \underline{S}_m$ $a_p = \bar{S}_o$ $S_m < S_o$ => l<p

Mai multe secvente de executie, o alegem pe cea cu cele mai putine niveluri

O secvență de execuție a unui sistem de n procese, C = (P,<) este orice şir α = a1 a2.....a2n de evenimente de inițiere şi terminare a proceselor cu respectarea relațiilor de precedență impuse de <.

c. Secvente partiale

$\alpha_p = a_1 a_2 \ldots a_k \quad k < 2n$

$\alpha_p$ - secventa partiala -> indica ce sarcini sunt active la un mom dat

$S_i$ este activa daca exista $a_j$, $a_j = \bar{S}_i$ , $j \leq k$ si nu exista $a_l = \underline{S}_i$ , $l \leq k$

O secvență de execuție parțială este orice prefix al unei secvențe de execuție.

## 18. Sistem de sarcini determinat

Un sistem format din procese care se execută în paralel şi cooperează pentru realizarea unor operaţii logice şi de calcul, şi produce acelaşi rezultat indiferent de durata de execuție a fiecărui proces independent, sau de ordinea în care acestea se execută, formează un sistem de procese funcțional, totaechival asincron (independent de viteza de execuție) sau sistem determinat.

Un sistem de procese este *nedeterminat* dacă rezultatele produse de procese independente depind de ordinea în care acestea se execută.

În Figura 5.2 se arată un exemplu de sistem nedeterminat:

$P_1$: $R_1 \leftarrow$ BUSFN (M; DCD (ADR))

$P_2$: M * DCD ( ADR ) $\leftarrow R_2$

$P_1$ - citeşte din locaţia de memorie de la adresa ADR;
$P_2$ - scrie în locaţia de memorie de la adresa ADR.

Nedeterminarea se poate rezolva introducând o relaţie de precedenţă adecvată între procesele care erau independente.

Pentru a stabili condiţiile necesare şi suficiente ca un sistem să fie determinat, vom considera un model simplificat în care sistemul fizic este privit ca o mulţime ordonată de locaţii de memorie:

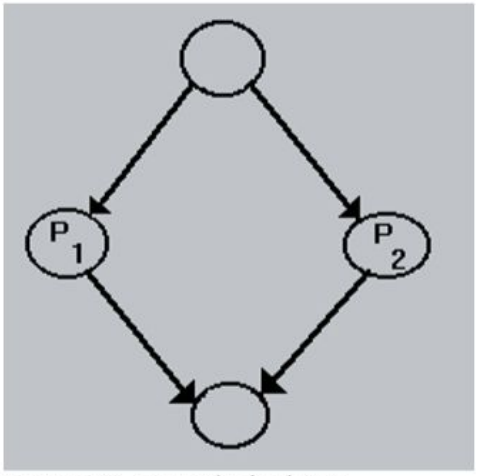



**Figura 5.2.** Exemplu de sistem nedeterminat

$\mathbf{M} = ( M_1, M_2, ....., M_m )$

ce conţin orice valoare dintr-o mulţime de valori **V**.

Stările sistemului vor fi definite de valorile care se găsesc în memorie la un moment dat:

De exemplu dacă:

$\alpha = a_1 a_2 ..... a_{2n}$ şi

$\sigma = s_0 s_1 ..... s_{2n}$

reprezintă o secvenţă de execuţie, respectiv secvenţa de stări corespunzătoare,
iar $M_i[k]$ reprezintă valoarea din celula $M_i$ imediat după evenimentul $a_k$; starea sistemului va fi:

$$s_k = [ M_1[k], M_2[k], ....., M_m[k] ]$$

Mulţimea tuturor stărilor poate fi definită astfel:

$\mathbf{\Sigma} = \mathbf{V}^m$ $\qquad$ $\mathbf{V}^m =$ $\qquad$ $[ \mathbf{V} \times \mathbf{V} \times ..... \times \mathbf{V} ]$, produs cartezian.

Pentru a formaliza efectul execuţiei unui proces asupra celulelor de memorie vom considera că fiecărui proces P i se asociază funcţia:

$$f_p : V^d \longrightarrow V^r$$

unde:

$d = |D_p|$ cardinalul domeniului valorilor de intrare, $D_p$;

$r = |R_p|$ cardinalul domeniului valorilor de ieşire, $R_p$.

Act
Go t

Pentru o stare iniţială $s_0$ şi o secvenţă de execuţie $\alpha$, secvenţa corespunzătoare de stări:

$\sigma = s_0 s_1 ..... s_{2n}$ este definită în felul următor :

Fie $D_p = ( M_{x1}, M_{x2}, ....., M_{xd} )$ domeniul valorilor de intrare;

$R_p = ( M_{y1}, M_{y2}, ....., M_{yr} )$ domeniul valorilor de ieşire;

1°. dacă $a_{k+1} = \overline{P}$, atunci $M_i[k+1] = M_i[k]$, $1 \le i \le m$;

2°. dacă $a_{k+1} = \underline{P}$, şi $a_l = \overline{P}$ , $1 \le k$ , atunci stările locaţiilor de memorie din domeniul de valori al lui P la momentul k+1 sunt:

$$[M_{y1}[k+1], M_{y2}[k+1], ..., M_{yr}[k+1]] = f_p[M_{x1}[l], M_{x2}[l], ..., M_{xd}[l]]$$

şi

$$M_i[k+1] = M_i[k] \quad \text{pentru } (\forall)\, M_i \notin R_p$$

Altfel spus, dacă $a_{k+1}$ este un eveniment de iniţiere nu apare o schimbare a stării, adică $s_{k+1} = s_k$.

Dacă însă $a_{k+1}$ este un eveniment de terminare a lui P, $s_{k+1} \neq s_k$ doar în domeniul Rs, noile valori din Rs fiind determinate de $f_p$ pe baza valorilor din domeniul de definiţie din momentul imediat precedent iniţierii lui P .

Secvenţa de stări $\sigma = s_0 s_1 ..... s_{2n}$ ce rezultă dintr-o secvenţă de execuţie, poate fi reprezentată sub forma unui tablou de dimensiuni m * (2n+1) având pe *linii* celulele de *memorie M*, iar pe *coloane stările*.

**19. Organizarea microoperatiilor. Avantaje + dezavantaje.**

a. Organizarea verticala
   i. Avantaj: codificare minima (dimensiune)
   ii. Dezavantaj:
      1. eliminarea controlului paralel asupra resurselor
      2. inflexibilitatea dezvoltării sau completării sistemului în ceea ce priveşte introducerea de noi microoperații.

b. Organizare orizontala
   i. Avantaj: paralelism maxim + flexibilitate mare
   ii. Dimensiune prea mare

c. Codificare minimala
   i. Combina flexibilitatea si paralelismul potențial oferite de codificarea orizontală cu eficiența codificării verticale

d. Codificare minimala pe 2 niveluri
   i. Mai putin eficienta decat cea minimala

e. Codificare cu control rezidual
   i. asigură o economie de memorie de control atunci când unele primitive funcționale realizează aceeaşi operație în mod repetat sau când un set de microoperații este activ o perioadă mare de timp, iar alte seturi de microoperații se modifică.

f. Codificarea cu control prin adrese
   i. Dezavantaj: necesită două accese la memorie în cadrul unui ciclu de microinstrucțiune
   ii. Avantaj: se realizează o economie importantă de memorie.

**20. Exemple de calculatoare paralele**

IBM's Blue Gene/P massively parallel supercomputer.
Blue Gene/L - eServer BlueGene Solution IBM
Roadrunner - BladeCenter Cluster IBM
Ranger - SunBlade - opteron quad 2Ghz infiniband- Sun Microsystems
Jaguar-Cray XT4 QuadCore 2.1Ghz - CrayInc.

**21. Tipuri de microinstructiuni**

Pentru unitatea de comandă microprogramată a calculatorului didactic vom defini două tipuri de microinstrucțiuni şi anume:

- Microinstructiune operationala, care specifica controlul primitivelor functionale ale unitatii de executie
- microinstrucțiune de ramificație, care permite testarea stării primitivelor funcționale şi asigură ramificația în microprogram .

**22.** Algoritmi de impartire pe niveluri(2)

**23. Legatura dintre algoritmi paraleli si arhitecturi paralele**

| ALGORITMI | ARHITECTURA |
|---|---|
| Granularitatea modulului de procesare(task, proces, thread, etc) | Complexitatea procesorului + comunicatia intre procesoare |
| Control concurent | Analiza modului de operare procesor(mono, pipeline, SIMD, MIMD, etc) |
| Mecanismul datelor<br>- Structurare<br>- Modalitate de acces | Structurarea memoriei |
| Geometria comunicatiei<br>- statica/dinamica<br>- simetrica/asimetrica | Retea de comutare<br>- Crossbar<br>- Delta $a^n x b^n$<br>- Trunchi k |
| Complexitate algoritm | Nr de procesoare / dimensiune memorie / performanta retea de comutare |

**24.** Un subiect la alegere(3)

**25.** Descrieti arhitectura din laborator - aici nu vrea cam ceea ce vrea si la urmatoarea intrebare?

intel xeon quad core
8 RAM
12 MB cache
2.5 GHz

## 26. Descrieti structura sistemului pe care ati lucrat la laborator(2)

### a. Si aici ce vrea mai exact? Procesor, memorie RAM, placa video? Si daca da, stie cineva care sunt?

National Center for Information Technology NCIT - s-a infiintat in 2001 atunci cand a aparut primul cluster al facultatii, **CoLaborator**. In 2006, infrastructura a fost extinsa ducand la aparitia celui de-al doilea cluster mult mai puternic decat CoLaborator, **NCIT**. Cele 2 clustere functioneaza ca un intreg datorita vitezii foarte mari dintre ele. (se planuieste ca aceasta viteza sa ajunga la 10Gb Ethernet)

Clusterul NCIT al facultății este accesibil prin front-end processor la adresa fep.grid.pub.ro folosind protocolul SSH.

Infrastructura de cloud din cadrul clusterului NCIT este bazata pe solutia opensource Openstack. Aceasta este o solutie de IaaS (Infrastructure as a Service).

In prezent, exista 6 arhitecturi diferite disponibile in cluster si anume:

- Intel Xeon Quad 64b
- Intel Xeon Nehalem 64b
- AMD Opteron 64b
- IBM Cell BE EDp 32/64b
- IBM Power7 64b
- NVidia Tesla M2070 32/64b

Tool-uri folosite in cluster pentru dezvoltare:

- Sun Studio
- Open MP
- Open MPI

Pentru debugging folosim:

- TotalView Debugger
- Sun Studio

Pentru profiling si analize de performanta:

- Sun Studio Performance Tools
- Intel VTune

**Configurație (hosturi și caracteristici)**

| | |
|---|---|
| `ibm-quad.q` | Intel Xeon, E5405, 2 GHz, IBM HS21, 28 nodes |
| `ibm-nehalem.q` | Intel Xeon, E5630, 2.53 GHz, IBM HS22, 4 nodes |
| `ibm-opteron.q` | AMD Opteron, IBM LS22, 14 nodes |
| `ibm-cell-qs22.q` | BECell Broadband, IBM QS22, 4 nodes |
| `ibm-dp.q` | 2 x 12-core Intel Xeon X5650 + 2 Nvidia Tesla, iDataPlex dx360 M3 Server |